VOLUMEN 03 · MÁSTER DATA & IA

SQL II

Subconsultas, CTE, funciones de ventana, fechas avanzadas y rendimiento. El SQL que se pregunta en las entrevistas y el que se usa en los informes de verdad.

PARA QUIÉN

Álvaro, con el Volumen 02 terminado y la capa silver construida y comprobada con tus seis `true`.

CÓMO ESTÁ ESCRITO

Igual que el Volumen 02: cada consulta se ejecuta de verdad sobre silver y su resultado aparece tal cual. Lo nuevo se explica despacio, con su trampa provocada y el error real.

INCLUYE

Nueve tickets de cinco departamentos con su solución y la revisión de Elena, entre ellos el que salió del MJD-208, una comparación de dialectos y una chuleta de funciones de ventana.

Preparado para Álvaro · Octubre de 2026 · Unos dos días y medio de trabajo

Índice

PARTE 0 · ANTES DE EMPEZAR 3

1. Cómo trabajar con este volumen 4

PARTE 1 · CONSULTAS QUE SE APOYAN EN OTRAS 6

2. Subconsultas 7

3. CTE a fondo 13

PARTE 2 · FUNCIONES DE VENTANA 19

4. Qué es una ventana 20

5. Rankings 24

6. Acumulados y medias móviles 29

7. Mirar atrás y adelante 33

PARTE 3 · FECHAS AVANZADAS 38

8. Calendarios y comparaciones 39

PARTE 4 · RENDIMIENTO Y DIALECTOS 46

9. Rendimiento 47

10. El mismo SQL en otros sitios 53

TICKETS · OTRA SEMANA EN MAREJADA 55

Tickets · Volumen 03 56

Soluciones · Volumen 03 59

APÉNDICES · PARA TENER A MANO 76

A. Chuleta de SQL II 77

B. Qué viene en el Volumen 04 79

PARTE 0

Antes de empezar

Desde dónde trabajas a partir de ahora y la consulta base que vas a reutilizar todo el volumen.

1. Cómo trabajar con este volumen

1. Cómo trabajar con este volumen

1.1 Ahora se trabaja sobre silver

En el Volumen 02 pasaste la mitad del tiempo desconfiando de bronze: duplicados, provincias escritas de quince formas, fechas de 1900. Todo eso lo resolviste una vez, en `sql/01_construir_silver.sql`, y lo comprobaste con `sql/02_comprobar_silver.sql`. A partir de aquí, **todas las consultas parten de silver**. Bronze solo se toca para comparar o para investigar un problema nuevo.

Antes de empezar, la comprobación de siempre. Si alguna sale `false`, para: el volumen entero se apoya en esto.

TÚ ESCRIBES

```
cd ~/master-data-ia
duckdb db/marejada.duckdb < sql/02_comprobar_silver.sql
```

SQL · EN DUCKDB

```
SELECT 'Clientes: misma cantidad de fichas que en bronze' AS comprobacion,
       (SELECT count(*) FROM silver.clientes) = (SELECT count(*) FROM bronze.clientes) AS ok
UNION ALL
SELECT 'Líneas: ningún (pedido, línea) repetido',
       (SELECT count(*) FROM (SELECT pedido_id, num_linea FROM silver.lineas_pedido
                              GROUP BY ALL HAVING count(*) > 1)) = 0
UNION ALL
SELECT 'Pedidos: 23 entregas marcadas como sospechosas',
       (SELECT count(*) FROM silver.pedidos WHERE entrega_sospechosa) = 23;
```

RESULTADO

```
┌──────────────────────────────────────────────────┬─────────┐
│                   comprobacion                   │   ok    │
│                     varchar                      │ boolean │
├──────────────────────────────────────────────────┼─────────┤
│ Clientes: misma cantidad de fichas que en bronze │ true    │
│ Líneas: ningún (pedido, línea) repetido          │ true    │
│ Pedidos: 23 entregas marcadas como sospechosas   │ true    │
└──────────────────────────────────────────────────┴─────────┘
```

Aquí se muestran tres de las seis para no ocupar media página; tú ejecuta el script entero.

1.2 Tu sesión de trabajo

La de siempre, con una carpeta nueva para los tickets de este volumen:

TÚ ESCRIBES

```
mkdir -p sql/03-sql-ii
duckdb db/marejada.duckdb
```

Y dentro de DuckDB, una costumbre nueva para este volumen, porque al final hablaremos de rendimiento:

TÚ ESCRIBES

```
.timer on
```

A partir de ese momento, debajo de cada resultado DuckDB te dice cuánto ha tardado (`Run Time (s): real 0.012 ...`). Casi todo tardará milésimas. Cuando algo tarde segundos, sabrás que hay algo que mirar.

1.3 La fecha de corte, otra vez

Los datos acaban el **30 de septiembre de 2026**. Todas las preguntas del tipo "los últimos tres meses" o "este año" se calculan desde `DATE '2026-09-30'`, nunca desde `current_date`. En este volumen vas a escribir muchas comparaciones de fechas y la tentación de usar `current_date` será mayor. No lo hagas: una consulta que da un resultado distinto cada día que la ejecutas no se puede revisar.

1.4 Qué cuenta como "venta" en este volumen

Para no repetirlo en cada ejemplo, el criterio por defecto:

CONCEPTO	CRITERIO EN SILVER
Una venta	Una línea de `silver.lineas_pedido` cuyo pedido no está cancelado
Su importe	`importe_neto`, ya calculado en silver (cantidad × precio × (1 − descuento))
Su fecha	`fecha` del pedido (mes de pedido, el criterio comercial de Íñigo)
Su cliente	`cliente_maestro_id`: las fichas duplicadas cuentan como un solo cliente

Cuando un ticket pida el criterio de contabilidad (mes de entrega, con portes), lo dirá, como en el MJD-204.

PARTE 1

Consultas que se apoyan en otras

Subconsultas y CTE: partir un problema grande en pasos pequeños que se pueden comprobar uno a uno.

2. Subconsultas

3. CTE a fondo

2. Subconsultas

2.1 Qué es una subconsulta

Una **subconsulta** es una consulta escrita dentro de otra, entre paréntesis. Ya las usaste sin ponerles nombre: el `(SELECT count(*) FROM ...)` de las comprobaciones de silver, o el `IN (SELECT pedido_id FROM bronze.pedidos)` que excluye las líneas huérfanas. Existen porque muchas preguntas se responden en dos tiempos: primero calculas algo y después lo usas.

"¿Qué productos cuestan más que la media del catálogo?" es una pregunta de dos tiempos: primero la media, después la comparación.

SQL · EN DUCKDB

```
SELECT round(avg(precio_lista), 2) AS precio_medio
FROM silver.productos;
```

RESULTADO

```
┌──────────────┐
│ precio_medio │
│    double    │
├──────────────┤
│        32.29 │
└──────────────┘
```

SQL · EN DUCKDB

```
SELECT sku, nombre, formato, precio_lista
FROM silver.productos
WHERE precio_lista > (SELECT avg(precio_lista) FROM silver.productos)
ORDER BY precio_lista DESC
LIMIT 5;
```

RESULTADO

```
┌─────────┬──────────────────────┬──────────────┬──────────────┐
│   sku   │        nombre        │   formato    │ precio_lista │
│ varchar │       varchar        │   varchar    │ decimal(10,2)│
├─────────┼──────────────────────┼──────────────┼──────────────┤
│ CER-0005 │ Faro Norte Especial │ Barril 30 L  │       117.77 │
│ CER-0055 │ Ola Fría Especial   │ Barril 30 L  │       116.55 │
│ CER-0042 │ Pejina Tostada      │ Barril 30 L  │       114.93 │
│ CER-0039 │ Pejina Especial     │ Barril 30 L  │       113.09 │
│ CER-0007 │ Faro Norte Tostada  │ Barril 30 L  │       106.97 │
└─────────┴──────────────────────┴──────────────┴──────────────┘
```

DuckDB ejecuta primero el paréntesis, obtiene un solo número (la media, 32,29 €) y después lo usa en el `WHERE` como si lo hubieras escrito a mano. La ventaja frente a escribir `WHERE precio_lista > 32.29` es que la consulta sigue siendo correcta mañana, cuando cambie el catálogo.

2.2 Los cuatro sitios donde puede ir

Una subconsulta puede devolver tres formas de resultado, y cada forma va en un sitio distinto:

DEVUELVE	SE LLAMA	DÓNDE SE USA	EJEMPLO
Un solo valor (una fila, una columna)	Escalar	Donde iría un número o un texto: en el `SELECT`, en el `WHERE`	`WHERE precio > (SELECT avg(...) ...)`

DEVUELVE	SE LLAMA	DÓNDE SE USA	EJEMPLO
Una columna con muchas filas	Lista	Detrás de `IN` o `NOT IN`	`WHERE pedido_id IN (SELECT ...)`
Una tabla	Tabla derivada	En el `FROM`, como si fuera una tabla	`FROM (SELECT ...) AS t`
Verdadero o falso	Existencial	Detrás de `EXISTS` o `NOT EXISTS`	`WHERE EXISTS (SELECT 1 ...)`

Si una subconsulta escalar devuelve más de una fila, no hay forma de usarla como un número, y DuckDB lo dice:

SQL · EN DUCKDB

```
SELECT sku, nombre
FROM silver.productos
WHERE precio_lista > (SELECT precio_lista FROM silver.productos WHERE categoria = 'Aguas');
```

ERROR

```
Invalid Input Error: More than one row returned by a subquery used as an expression - scalar subqueries
can only return a single row.
Use "SET scalar_subquery_error_on_multiple_rows=false" to revert to previous behavior of returning a
random row.
```

Hay 15 aguas, así que el paréntesis devuelve 15 precios. ¿Mayor que cuál de ellos? El mensaje lo dice con precisión: una subconsulta usada como valor (*scalar subquery*) solo puede devolver una fila. La segunda línea del error ofrece un ajuste para volver al comportamiento antiguo, que era devolver **una fila cualquiera** sin avisar. No lo uses nunca: es cambiar un error visible por un resultado aleatorio. La salida correcta es decir qué querías: si era "más caro que el agua más cara", agrega dentro del paréntesis (`SELECT max(precio_lista) ...`).

2.3 IN y la trampa de NOT IN

Paula quiere saber cuántos clientes no han hecho nunca un pedido por la web. Parece un `NOT IN` de manual: los clientes cuyo identificador no está en la lista de clientes que han comprado por la web.

SQL · EN DUCKDB

```
SELECT count(*) AS clientes_sin_web
FROM silver.clientes
WHERE NOT es_ficha_duplicada
  AND cliente_id NOT IN (SELECT cliente_maestro_id
                         FROM silver.pedidos
                         WHERE canal = 'Web B2B');
```

RESULTADO

```
┌──────────────────┐
│ clientes_sin_web │
│      int64       │
├──────────────────┤
│                0 │
└──────────────────┘
```

Cero. Todos los clientes de Marejada habrían comprado alguna vez por la web, incluidos los que se dieron de baja en 2024, antes de que la web existiera. Es imposible, y no ha salido ningún error.

La culpa es de un `NULL`. En silver hay pedidos web de clientes borrados del maestro, y esos pedidos tienen `cliente_maestro_id` vacío:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos_web_sin_cliente
FROM silver.pedidos
WHERE canal = 'Web B2B' AND cliente_maestro_id IS NULL;
```

RESULTADO

```
┌──────────────────────────┐
│ pedidos_web_sin_cliente  │
│          int64           │
├──────────────────────────┤
│                        3 │
└──────────────────────────┘
```

Tres pedidos bastan. `x NOT IN (1, 2, NULL)` significa `x <> 1 AND x <> 2 AND x <> NULL`, y `x <> NULL` es `NULL` (el "no se sabe" del Volumen 02, 4.6). Un `AND` en el que una parte es "no se sabe" nunca puede ser verdadero, así que el `WHERE` no deja pasar a nadie.

EXPRESIÓN	RESULTADO	POR QUÉ
`5 IN (1, 5, NULL)`	true	Encuentra el 5: basta con uno que coincida
`7 IN (1, 5, NULL)`	NULL	No encuentra el 7, pero "¿y si el NULL fuera un 7?"
`7 NOT IN (1, 5)`	true	Distinto de todos
`7 NOT IN (1, 5, NULL)`	NULL	"¿Distinto del NULL? No se sabe." El `WHERE` lo descarta

2.4 EXISTS, la versión segura

`NOT EXISTS` hace la misma pregunta de otra forma: "¿existe algún pedido web de este cliente?". No compara contra una lista, sino que busca filas, y una fila con `NULL` simplemente no es de este cliente:

SQL · EN DUCKDB

```
SELECT count(*) AS clientes_sin_web
FROM silver.clientes AS c
WHERE NOT c.es_ficha_duplicada
  AND NOT EXISTS (SELECT 1
                  FROM silver.pedidos AS p
                  WHERE p.canal = 'Web B2B'
                    AND p.cliente_maestro_id = c.cliente_id);
```

RESULTADO

```
┌─────────────────┐
│ clientes_sin_web │
│      int64       │
├─────────────────┤
│              88 │
└─────────────────┘
```

88 clientes, que es lo razonable.

DESMONTANDO EL COMANDO

`NOT EXISTS (SELECT 1 FROM silver.pedidos AS p WHERE ... AND p.cliente_maestro_id = c.cliente_id)`

`EXISTS ( ... )`	Verdadero si la subconsulta devuelve al menos una fila; falso si no devuelve ninguna. Nunca `NULL`.
`SELECT 1`	Lo que se selecciona da igual: `EXISTS` solo mira si hay filas. `SELECT 1` es la convención para dejarlo claro.
`p.cliente_maestro_id = c.cliente_id`	La subconsulta usa una columna de la consulta de fuera (`c`). Se dice que está **correlacionada**: conceptualmente, se evalúa una vez por cada cliente.
`NOT c.es_ficha_duplicada`	Contamos clientes reales, no fichas. Las fichas duplicadas no tienen pedidos propios en `cliente_maestro_id`.

IDEA CLAVE

Para "lo que no está en otra tabla", usa `NOT EXISTS` o el anti-join del Volumen 02 (8.4). `NOT IN` solo es seguro si estás completamente seguro de que la lista no tiene nulos, y "completamente seguro" hoy no lo es mañana.

SQL · EN DUCKDB

```
SELECT count(*) AS clientes_sin_web
FROM silver.clientes AS c
LEFT JOIN (SELECT DISTINCT cliente_maestro_id
           FROM silver.pedidos
           WHERE canal = 'Web B2B') AS w
       ON w.cliente_maestro_id = c.cliente_id
WHERE NOT c.es_ficha_duplicada
  AND w.cliente_maestro_id IS NULL;
```

RESULTADO

```
┌──────────────────┐
│ clientes_sin_web │
│      int64       │
├──────────────────┤
│               88 │
└──────────────────┘
```

El anti-join da lo mismo: 88. Dos caminos distintos, la misma cifra. Es la regla de "comprobar por otra vía" del máster.

2.5 Subconsultas correlacionadas

La de `NOT EXISTS` era correlacionada: usaba `c.cliente_id`, de la consulta de fuera. También se pueden poner en el `SELECT`. La fecha del último pedido de cada cliente, al lado de su nombre:

SQL · EN DUCKDB

```
SELECT c.cliente_id,
       c.nombre_comercial,
       (SELECT max(p.fecha)
        FROM silver.pedidos AS p
        WHERE p.cliente_maestro_id = c.cliente_id
          AND p.estado <> 'Cancelado') AS ultimo_pedido
FROM silver.clientes AS c
WHERE NOT c.es_ficha_duplicada
ORDER BY c.cliente_id
LIMIT 5;
```

RESULTADO

cliente_id	nombre_comercial	ultimo_pedido
int64	varchar	date
10001	Bar La Ola	2026-09-25
10002	Bar La Ría	2026-09-10
10003	Bar Bitácora	2026-07-30
10004	Cafetería El Llagar	2026-09-07
10005	Hotel La Castellana	2026-09-30

Se lee de forma muy natural ("para cada cliente, busca su último pedido"), y por eso es tan común entre quien empieza. El problema es que conceptualmente se ejecuta **una vez por fila**: 1.175 clientes, 1.175 búsquedas. DuckDB es listo y por dentro la reescribe como un `JOIN`, pero no todos los motores lo hacen, y en el capítulo 9 medirás la diferencia. En el capítulo 7 verás tres formas mejores de sacar lo mismo.

2.6 Tablas derivadas

Una subconsulta en el `FROM` se trata como una tabla más. Necesita un alias. ¿Cuántos clientes hay según el número de pedidos que han hecho en 2025?

SQL · EN DUCKDB

```
SELECT CASE
           WHEN pedidos < 12  THEN '1. Menos de 1 al mes'
           WHEN pedidos < 52  THEN '2. Entre 1 al mes y 1 a la semana'
           ELSE                    '3. Más de 1 a la semana'
       END AS frecuencia,
       count(*) AS clientes
FROM (SELECT cliente_maestro_id, count(*) AS pedidos
      FROM silver.pedidos
      WHERE estado <> 'Cancelado'
        AND fecha >= '2025-01-01' AND fecha < '2026-01-01'
        AND cliente_maestro_id IS NOT NULL
      GROUP BY cliente_maestro_id) AS por_cliente
GROUP BY frecuencia
ORDER BY frecuencia;
```

RESULTADO

frecuencia	clientes
varchar	int64
1. Menos de 1 al mes	106
2. Entre 1 al mes y 1 a la semana	850
3. Más de 1 a la semana	35

Agregar dos veces (primero por cliente, luego por tramo) es imposible en una sola consulta, porque un `GROUP BY` no puede usar el resultado de otro `GROUP BY`. La tabla derivada lo resuelve. Pero en cuanto hay dos o tres niveles, los paréntesis anidados se vuelven ilegibles. Para eso existen las CTE.

3. CTE a fondo

3.1 La misma consulta, de arriba abajo

Una **CTE** (*Common Table Expression*) es una subconsulta con nombre, declarada al principio con `WITH`. La viste en el Volumen 02 para evitar la trampa del abanico. La consulta anterior, con CTE:

SQL · EN DUCKDB

```
WITH pedidos_2025 AS (
    SELECT cliente_maestro_id, count(*) AS pedidos
    FROM silver.pedidos
    WHERE estado <> 'Cancelado'
      AND fecha >= '2025-01-01' AND fecha < '2026-01-01'
      AND cliente_maestro_id IS NOT NULL
    GROUP BY cliente_maestro_id
),
con_tramo AS (
    SELECT *,
           CASE
               WHEN pedidos < 12 THEN '1. Menos de 1 al mes'
               WHEN pedidos < 52 THEN '2. Entre 1 al mes y 1 a la semana'
               ELSE                   '3. Más de 1 a la semana'
           END AS frecuencia
    FROM pedidos_2025
)
SELECT frecuencia, count(*) AS clientes, round(avg(pedidos), 1) AS pedidos_medios
FROM con_tramo
GROUP BY frecuencia
ORDER BY frecuencia;
```

RESULTADO

frecuencia varchar	clientes int64	pedidos_medios double
1. Menos de 1 al mes	106	6.3
2. Entre 1 al mes y 1 a la semana	850	28.5
3. Más de 1 a la semana	35	64.2

Mismo resultado, más una columna, y se lee de arriba abajo como una receta: primero los pedidos de 2025 por cliente, después su tramo, al final el resumen. Cada paso tiene un nombre que explica lo que hace.

DESMONTANDO LA CONSULTA

`WITH a AS ( ... ), b AS ( ... ) SELECT ... FROM b`

`WITH`	Abre la lista de CTE. Solo se escribe una vez, al principio.
`a AS ( ... )`	Una CTE: un nombre y la consulta que lo define. Dentro va una consulta completa.
`, b AS ( ... )`	La siguiente CTE, separada por una coma. Puede usar las anteriores (`FROM a`), nunca las de después.
`SELECT ... FROM b`	La consulta final, la única que devuelve resultado. Sin ella, la CTE no hace nada.

LA COMA Y EL PUNTO Y COMA

Los dos errores más comunes al escribir CTE: poner `WITH` delante de cada una (solo va en la primera) y olvidar la coma entre ellas. Y uno más sutil: un `;` después del último paréntesis cierra la consulta antes del `SELECT` final, y DuckDB protesta porque el `WITH` se ha quedado sin consulta.

3.2 Depurar paso a paso

La gran ventaja de las CTE es que cada paso se puede comprobar por separado. Si el resultado final no te cuadra, cambias la consulta final por `SELECT * FROM el_paso_que_dudas` y miras qué sale:

SQL · EN DUCKDB

```
WITH pedidos_2025 AS (
    SELECT cliente_maestro_id, count(*) AS pedidos
    FROM silver.pedidos
    WHERE estado <> 'Cancelado'
      AND fecha >= '2025-01-01' AND fecha < '2026-01-01'
      AND cliente_maestro_id IS NOT NULL
    GROUP BY cliente_maestro_id
)
SELECT count(*) AS clientes, sum(pedidos) AS pedidos, min(pedidos) AS minimo, max(pedidos) AS maximo
FROM pedidos_2025;
```

RESULTADO

```
┌──────────┬──────────┬─────────┬─────────┐
│ clientes │ pedidos  │ minimo  │ maximo  │
│  int64   │ int128   │ int64   │ int64   │
├──────────┼──────────┼─────────┼─────────┤
│      991 │    27144 │       1 │     116 │
└──────────┴──────────┴─────────┴─────────┘
```

Los clientes del paso cuadran con la suma del resumen de 3.1, y los pedidos con lo que esperarías de un año. Es como poner un `console.log` a mitad de un programa, y es la forma profesional de no perderse en una consulta de cien líneas.

3.3 Una CTE se puede usar dos veces

Una tabla derivada solo existe en el sitio donde la escribes. Una CTE se puede usar las veces que haga falta. ¿Qué peso tiene cada canal en las ventas de 2025?

SQL · EN DUCKDB

```
WITH ventas_canal AS (
    SELECT p.canal, sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha >= '2025-01-01' AND p.fecha < '2026-01-01'
    GROUP BY p.canal
)
SELECT canal,
       round(ventas) AS ventas,
       round(100 * ventas / (SELECT sum(ventas) FROM ventas_canal), 1) AS pct
FROM ventas_canal
ORDER BY ventas DESC;
```

RESULTADO

```
┌────────────┬────────────┬────────┐
│   canal    │   ventas   │  pct   │
│  varchar   │   double   │ double │
├────────────┼────────────┼────────┤
│ Comercial  │ 9961686.0  │   63.8 │
│ Teléfono   │ 4332802.0  │   27.7 │
│ Web B2B    │ 1324889.0  │    8.5 │
└────────────┴────────────┴────────┘
```

`ventas_canal` aparece en el `FROM` y otra vez dentro de la subconsulta del total. Sin la CTE habría que escribir dos veces la misma unión de pedidos y líneas, con el riesgo de que un día alguien cambie una y no la otra. (En el capítulo 4 verás una forma todavía más corta de calcular un porcentaje del total.)

3.4 CTE, vista o tabla

Las tres guardan "una consulta con nombre", pero duran distinto:

QUÉ	CUÁNTO DURA	CUÁNDO USARLA
CTE (`WITH`)	Lo que dura la consulta	Pasos intermedios de una consulta concreta
Vista (`CREATE VIEW`)	Hasta que la borres. Guarda la **consulta**, no los datos: cada vez que la usas, se ejecuta de nuevo	Una consulta que vas a reutilizar en muchos sitios y que siempre debe estar al día
Tabla (`CREATE TABLE ... AS`)	Hasta que la borres. Guarda los **datos** del momento en que se creó	Resultados que no cambian, o que son caros de calcular. Silver es eso

Para probar una vista sin dejarla en tu base de datos, DuckDB permite crearla **temporal**: desaparece al cerrar la sesión.

SQL · EN DUCKDB

```
CREATE OR REPLACE TEMP VIEW ventas_validas AS
SELECT p.pedido_id, p.fecha, p.cliente_maestro_id, p.comercial_id, p.canal,
       l.producto_id, l.cantidad, l.importe_neto
FROM silver.pedidos AS p
JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
WHERE p.estado <> 'Cancelado';

SELECT count(*) AS lineas, round(sum(importe_neto)) AS ventas
FROM ventas_validas;
```

RESULTADO

```
┌─────────┬──────────────┐
│ lineas  │    ventas    │
│ int64   │    double    │
├─────────┼──────────────┤
│ 366928  │ 42633273.0   │
└─────────┴──────────────┘
```

`ventas_validas` ya no existe cuando cierras DuckDB. En el Volumen 04 harás lo contrario: tablas permanentes en una capa nueva, `gold`, pensadas para Power BI.

> **NOTA**
>
> En el resto del volumen escribimos la unión de pedidos y líneas cada vez, para que cada consulta funcione sola si la copias. En tu trabajo, una vista temporal como esta te ahorra escribirla una y otra vez durante la sesión.

3.5 CTE recursivas: recorrer una jerarquía

Hay una pregunta que ninguna de las herramientas anteriores sabe responder: ¿quién depende, directa o indirectamente, de Íñigo? Con un autojoin (Volumen 02, 8.5) llegas al jefe directo. Con dos, al jefe del jefe. Pero no sabes cuántos niveles hay, y no puedes escribir un número de joins que no conoces.

Una **CTE recursiva** se llama a sí misma hasta que no encuentra más filas:

SQL · EN DUCKDB

```
WITH RECURSIVE jerarquia AS (
    -- 1. Punto de partida: quien no tiene jefe
    SELECT comercial_id, nombre_completo, 1 AS nivel,
           nombre_completo AS ruta
    FROM silver.comerciales
    WHERE jefe_id IS NULL

    UNION ALL

    -- 2. Paso: los que tienen como jefe a alguien que ya está en la lista
    SELECT c.comercial_id, c.nombre_completo, j.nivel + 1,
           j.ruta || ' > ' || c.nombre_completo
    FROM silver.comerciales AS c
    JOIN jerarquia AS j ON c.jefe_id = j.comercial_id
)
SELECT nivel, ruta
FROM jerarquia
ORDER BY ruta;
```

RESULTADO

nivel int32	ruta varchar
1	Íñigo Sainz Ruiz
2	Íñigo Sainz Ruiz > Nuria Álvarez Menéndez
3	Íñigo Sainz Ruiz > Nuria Álvarez Menéndez > Adrián Suárez Fernández
3	Íñigo Sainz Ruiz > Nuria Álvarez Menéndez > Ane Etxeberria Zubiri
3	Íñigo Sainz Ruiz > Nuria Álvarez Menéndez > Mikel Goikoetxea Arana
2	Íñigo Sainz Ruiz > Raquel Fernández Cobo
3	Íñigo Sainz Ruiz > Raquel Fernández Cobo > Celia Gómez Sáiz
3	Íñigo Sainz Ruiz > Raquel Fernández Cobo > Daniel Gutiérrez Pérez
3	Íñigo Sainz Ruiz > Raquel Fernández Cobo > Héctor Bustamante Lavín
3	Íñigo Sainz Ruiz > Raquel Fernández Cobo > Lorena Arce Díaz
3	Íñigo Sainz Ruiz > Raquel Fernández Cobo > Pablo Revuelta Gómez
3	Íñigo Sainz Ruiz > Raquel Fernández Cobo > Sara Ceballos Ortiz
2	Íñigo Sainz Ruiz > Rubén Martín Calvo
2	Íñigo Sainz Ruiz > Óscar Peña Revilla

14 rows — 2 columns

DESMONTANDO LA CONSULTA

`WITH RECURSIVE jerarquia AS ( inicio UNION ALL paso )`

RECURSIVE	Avisa de que la CTE se usará a sí misma. Obligatorio en DuckDB, Postgres y Spark; en SQL Server no se escribe.
inicio	La primera consulta, la que no se usa a sí misma: las filas de partida (Íñigo, nivel 1).
UNION ALL	Une las filas de partida con las que va añadiendo cada vuelta.
paso	La consulta que usa `jerarquia`: en cada vuelta, busca a quienes tienen como jefe a alguien encontrado en la vuelta anterior.
fin	Cuando una vuelta no encuentra a nadie nuevo, para. Si los datos tuvieran un ciclo (A jefe de B y B jefe de A), no pararía nunca.

Tres niveles: Íñigo; quienes dependen directamente de él (las dos jefas de ventas y los dos comerciales de Castilla, Rubén y Óscar); y los comerciales de cada jefa. En Marejada es pequeño, pero el patrón es el mismo para un organigrama de diez mil personas, para las categorías y subcategorías de un catálogo o para una lista de materiales.

3.6 Generar filas que no existen

La recursión también sirve para fabricar filas: por ejemplo, todos los meses entre dos fechas, aunque en alguno no haya ventas. Lo necesitarás en el capítulo 8. Con recursión:

SQL · EN DUCKDB

```
WITH RECURSIVE meses AS (
    SELECT DATE '2026-01-01' AS mes
    UNION ALL
    SELECT mes + INTERVAL 1 MONTH
    FROM meses
    WHERE mes < DATE '2026-09-01'
)
SELECT mes FROM meses;
```

RESULTADO

```
┌────────────┐
│    mes     │
│    date    │
├────────────┤
│ 2026-01-01 │
│ 2026-02-01 │
│ 2026-03-01 │
│ 2026-04-01 │
│ 2026-05-01 │
│ 2026-06-01 │
│ 2026-07-01 │
│ 2026-08-01 │
│ 2026-09-01 │
└────────────┘
```

Funciona, pero hay una forma mucho más corta que verás en el capítulo 8, `generate_series`. Lo importante de este ejemplo es la condición `WHERE mes < ...`: es la que hace que la recursión pare.

IDEA CLAVE

Una CTE es una subconsulta con nombre. Úsalas para escribir de arriba abajo, comprobar cada paso por separado y no repetir código. Las recursivas recorren jerarquías y generan series; las vistas son CTE que sobreviven a la consulta.

PARTE 2

Funciones de ventana

Calcular sobre un grupo de filas sin perder el detalle de cada una: porcentajes, rankings, acumulados y comparaciones con la fila anterior.

4. Qué es una ventana

5. Rankings

6. Acumulados y medias móviles

7. Mirar atrás y adelante

4. Qué es una ventana

4.1 El problema que resuelven

`GROUP BY` resume: de muchas filas hace una. Muchas veces quieres otra cosa: **conservar cada fila** y, al lado, un cálculo sobre un grupo de filas. El peso de cada canal en el total, la posición de cada producto en su categoría, las ventas de cada mes junto a las del mes anterior. Con lo que sabes hasta ahora, cada una de esas preguntas necesita una subconsulta o un autojoin. Con una **función de ventana**, es una columna más.

Compara. Con `GROUP BY` obtienes el total por canal, y las filas de detalle desaparecen:

SQL · EN DUCKDB

```
SELECT canal, count(*) AS pedidos
FROM silver.pedidos
WHERE fecha >= '2026-09-29'
GROUP BY canal
ORDER BY canal;
```

RESULTADO

```
┌────────────┬─────────┐
│   canal    │ pedidos │
│  varchar   │  int64  │
├────────────┼─────────┤
│ Comercial  │     107 │
│ Teléfono   │      51 │
│ Web B2B    │      63 │
└────────────┴─────────┘
```

Con una ventana, cada pedido sigue en su fila, y la columna nueva lleva el recuento de su canal:

SQL · EN DUCKDB

```
SELECT pedido_id, fecha, canal,
       count(*) OVER (PARTITION BY canal) AS pedidos_del_canal
FROM silver.pedidos
WHERE fecha >= '2026-09-29'
ORDER BY canal, pedido_id;
```

RESULTADO

```
┌────────────┬────────────┬────────────┬──────────────────┐
│ pedido_id  │   fecha    │   canal    │ pedidos_del_canal │
│   int64    │    date    │  varchar   │       int64       │
├────────────┼────────────┼────────────┼──────────────────┤
│ 2026024959 │ 2026-09-29 │ Comercial  │              107 │
│ 2026024960 │ 2026-09-29 │ Comercial  │              107 │
│ 2026024964 │ 2026-09-29 │ Comercial  │              107 │
│ 2026024965 │ 2026-09-29 │ Comercial  │              107 │
│      .     │     .      │     .      │                . │
│      .     │     .      │     .      │                . │
│      .     │     .      │     .      │                . │
│ 2026025160 │ 2026-09-30 │ Web B2B    │               63 │
│ 2026025166 │ 2026-09-30 │ Web B2B    │               63 │
│ 2026025167 │ 2026-09-30 │ Web B2B    │               63 │
│ 2026025168 │ 2026-09-30 │ Web B2B    │               63 │
└────────────┴────────────┴────────────┴──────────────────┘
  221 rows (8 shown)                          4 columns
```

El resultado tiene tantas filas como pedidos, no como canales. Es lo que distingue a una función de ventana: **calcula sobre un grupo, pero no lo colapsa**.

DESMONTANDO LA CONSULTA

`count(*) OVER (PARTITION BY canal)`

`count(*)`	La función: puede ser cualquier agregado (`sum`, `avg`, `min`, `max`, `count`) o una de las funciones propias de ventana del capítulo 5.
`OVER ( ... )`	Lo que la convierte en función de ventana. Dentro se dice sobre qué filas calcular. `OVER ()` vacío significa "sobre todas las filas del resultado".
`PARTITION BY canal`	Parte las filas en grupos, como un `GROUP BY` que no colapsa. Cada fila ve solo las de su grupo.

4.2 El porcentaje del total en una línea

El peso de cada canal en las ventas de 2025, que en el 3.3 necesitó una CTE usada dos veces:

SQL · EN DUCKDB

```
SELECT p.canal,
       round(sum(l.importe_neto)) AS ventas,
       round(100 * sum(l.importe_neto) / sum(sum(l.importe_neto)) OVER (), 1) AS pct
FROM silver.pedidos AS p
JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
WHERE p.estado <> 'Cancelado'
  AND p.fecha >= '2025-01-01' AND p.fecha < '2026-01-01'
GROUP BY p.canal
ORDER BY ventas DESC;
```

RESULTADO

```
┌────────────┬──────────┬─────────┐
│   canal    │  ventas  │   pct   │
│  varchar   │  double  │ double  │
├────────────┼──────────┼─────────┤
│ Comercial  │ 9961686.0 │   63.8 │
│ Teléfono   │ 4332802.0 │   27.7 │
│ Web B2B    │ 1324889.0 │    8.5 │
└────────────┴──────────┴─────────┘
```

Las mismas cifras que en el 3.3. El truco está en `sum(sum(l.importe_neto)) OVER ()`, que parece un error y no lo es. Se lee de dentro afuera: el `sum` interior es el agregado normal del `GROUP BY` (las ventas de cada canal); el exterior, con `OVER ()`, suma esas tres cifras de los tres canales. Funciona porque las ventanas se calculan **después** del `GROUP BY`, sobre sus filas ya agrupadas.

4.3 Dónde va la ventana en el orden de ejecución

La tabla del Volumen 02 (5.3), con la pieza nueva:

ORDEN	CLÁUSULA	QUÉ HACE
1	FROM y JOIN	Elige las tablas
2	WHERE	Filtra filas
3	GROUP BY	Agrupa
4	HAVING	Filtra grupos
5	**Funciones de ventana**	Calculan sobre las filas que han sobrevivido a todo lo anterior

ORDEN	CLÁUSULA	QUÉ HACE
6	QUALIFY	Filtra por el resultado de una ventana (4.4)
7	SELECT	Pone nombres
8	DISTINCT, ORDER BY, LIMIT	Como siempre

Dos consecuencias. La primera: una ventana ve las filas **después** del `WHERE`. Si filtras septiembre, el "total" de `OVER ()` es el de septiembre, no el de todo el año. La segunda, que provoca el error más común de este capítulo: no se puede usar una ventana en el `WHERE`, porque el `WHERE` se ejecuta antes de que exista.

SQL · EN DUCKDB

```
SELECT pedido_id, fecha, cliente_maestro_id
FROM silver.pedidos
WHERE row_number() OVER (PARTITION BY cliente_maestro_id ORDER BY fecha DESC) = 1;
```

ERROR

```
Binder Error: WHERE clause cannot contain window functions!
```

El mensaje lo dice tal cual: las funciones de ventana no se permiten en el `WHERE`.

4.4 QUALIFY: el WHERE de las ventanas

Para filtrar por una ventana hay dos formas. La universal: calcularla en una CTE y filtrar fuera. La corta: `QUALIFY`, que es a las ventanas lo que `HAVING` es a los agregados.

SQL · EN DUCKDB

```
SELECT pedido_id, fecha, cliente_maestro_id
FROM silver.pedidos
WHERE cliente_maestro_id IN (10001, 10002, 10003)
QUALIFY row_number() OVER (PARTITION BY cliente_maestro_id ORDER BY fecha_pedido DESC) = 1
ORDER BY cliente_maestro_id;
```

RESULTADO

```
┌─────────────┬────────────┬──────────────────┐
│ pedido_id   │   fecha    │ cliente_maestro_id │
│   int64     │    date    │       int64        │
├─────────────┼────────────┼──────────────────┤
│ 2026024707  │ 2026-09-25 │            10001 │
│ 2026023331  │ 2026-09-10 │            10002 │
│ 2026018609  │ 2026-07-30 │            10003 │
└─────────────┴────────────┴──────────────────┘
```

El último pedido de cada uno de esos tres clientes. `row_number()` numera los pedidos de cada cliente del más reciente al más antiguo (lo verás en el 5.1), y `QUALIFY ... = 1` se queda con el primero de cada uno.

QUALIFY NO ESTÁ EN TODAS PARTES

`QUALIFY` existe en DuckDB, Databricks y Snowflake, pero **no** en SQL Server ni en el Warehouse de Fabric. Allí se escribe la versión con CTE: calcular la ventana con un nombre (`AS rn`) y filtrar después con `WHERE rn = 1`. Saber escribir las dos te va a hacer falta en el DP-600.

SQL · EN DUCKDB

```
WITH numerados AS (
    SELECT pedido_id, fecha, cliente_maestro_id,
           row_number() OVER (PARTITION BY cliente_maestro_id ORDER BY fecha_pedido DESC) AS rn
    FROM silver.pedidos
    WHERE cliente_maestro_id IN (10001, 10002, 10003)
)
SELECT pedido_id, fecha, cliente_maestro_id
FROM numerados
WHERE rn = 1
ORDER BY cliente_maestro_id;
```

RESULTADO

pedido_id	fecha	cliente_maestro_id
int64	date	int64
2026024707	2026-09-25	10001
2026023331	2026-09-10	10002
2026018609	2026-07-30	10003

4.5 Ponerle nombre a una ventana

Cuando varias columnas usan la misma ventana, se declara una vez con `WINDOW` y se reutiliza por su nombre. Es la misma idea que una CTE:

SQL · EN DUCKDB

```
SELECT canal,
       pedido_id,
       count(*) OVER w             AS pedidos_canal,
       round(avg(gastos_envio) OVER w, 2) AS portes_medios_canal
FROM silver.pedidos
WHERE fecha = '2026-09-30'
WINDOW w AS (PARTITION BY canal)
ORDER BY canal, pedido_id
LIMIT 6;
```

RESULTADO

canal	pedido_id	pedidos_canal	portes_medios_canal
varchar	int64	int64	double
Comercial	2026025077	49	3.43
Comercial	2026025079	49	3.43
Comercial	2026025083	49	3.43
Comercial	2026025084	49	3.43
Comercial	2026025085	49	3.43
Comercial	2026025086	49	3.43

5. Rankings

5.1 Tres formas de numerar

¿Cuáles fueron los clientes con más pedidos en 2025? Hay tres funciones para poner un número de posición, y se diferencian solo en cómo tratan los **empates**:

SQL · EN DUCKDB

```
WITH pedidos_2025 AS (
    SELECT cliente_maestro_id, count(*) AS pedidos
    FROM silver.pedidos
    WHERE estado <> 'Cancelado'
      AND fecha >= '2025-01-01' AND fecha < '2026-01-01'
      AND cliente_maestro_id IS NOT NULL
    GROUP BY cliente_maestro_id
)
SELECT c.nombre_comercial,
       t.pedidos,
       row_number() OVER w AS row_number,
       rank()       OVER w AS rank,
       dense_rank() OVER w AS dense_rank
FROM pedidos_2025 AS t
JOIN silver.clientes AS c ON c.cliente_id = t.cliente_maestro_id
WINDOW w AS (ORDER BY t.pedidos DESC)
ORDER BY t.pedidos DESC, c.nombre_comercial
LIMIT 10;
```

RESULTADO

nombre_comercial varchar	pedidos int64	row_number int64	rank int64	dense_rank int64
Supermercado La Sidra	116	1	1	1
Supermercado Los Arcos	101	2	2	2
Supermercado La Rula	85	3	3	3
Supermercado Los Pinares	81	4	4	4
Bar Salitre	73	6	5	5
Supermercado El Mercado	73	5	5	5
Supermercado La Fuente	72	7	7	6
Restaurante El Pasiego	71	8	8	7
Hotel La Taberna	67	10	9	8
Restaurante Los Arcos	67	9	9	8

10 rows — 5 columns

Mira las dos parejas empatadas, con 73 y con 67 pedidos:

FUNCIÓN	CON EMPATE	DESPUÉS DEL EMPATE	ÚSALA PARA
`row_number()`	Números distintos, **sin criterio**: uno de los dos se lleva el 5 y el otro el 6	Sigue: 7	Quedarte con exactamente una fila por grupo
`rank()`	Los dos el mismo puesto (5)	**Salta** al 7, como en una carrera: no hay sexto	Rankings "deportivos"
`dense_rank()`	Los dos el mismo puesto (5)	**No salta**: 6	"Los 3 mejores niveles", aunque haya varios por nivel

ROW_NUMBER CON EMPATES ES UNA LOTERÍA

Con empates, `row_number()` reparte los números como le viene mejor al motor, y puede cambiar de una ejecución a otra o entre DuckDB y Fabric. En el resultado de arriba, Supermercado El Mercado tiene el 5 y Bar Salitre el 6, pero el `ORDER BY` final los muestra en orden alfabético: no hay ninguna relación entre las dos cosas. Si usas `row_number()` para elegir una fila, añade siempre un desempate que no pueda repetirse: `ORDER BY pedidos DESC, cliente_maestro_id`.

5.2 El top N de cada grupo

La pregunta que más se hace con rankings, y una de las favoritas de las entrevistas: los tres productos más vendidos **de cada categoría**. Un `ORDER BY ... LIMIT 3` da los tres primeros de todo el catálogo; hace falta numerar dentro de cada categoría con `PARTITION BY`:

SQL · EN DUCKDB

```
WITH ventas_producto AS (
    SELECT pr.categoria, pr.sku, pr.nombre, pr.formato,
           sum(l.importe_neto) AS ventas
    FROM silver.lineas_pedido AS l
    JOIN silver.pedidos   AS p  ON p.pedido_id    = l.pedido_id
    JOIN silver.productos AS pr ON pr.producto_id = l.producto_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha >= '2025-01-01' AND p.fecha < '2026-01-01'
    GROUP BY pr.categoria, pr.sku, pr.nombre, pr.formato
)
SELECT categoria, sku, nombre, round(ventas) AS ventas,
       rank() OVER (PARTITION BY categoria ORDER BY ventas DESC) AS puesto
FROM ventas_producto
QUALIFY puesto <= 3
ORDER BY categoria, puesto;
```

RESULTADO

categoria varchar	sku varchar	nombre varchar	ventas double	puesto int64
Aguas	AGU-0001	Agua Fuente Pas	154829.0	1
Aguas	AGU-0015	Agua Aguas de Liérganes con gas	124332.0	2
Aguas	AGU-0004	Agua Fuente Pas	54300.0	3
Alimentación	ALI-0001	Anchoas de Santoña en aceite	1065599.0	1
Alimentación	ALI-0015	Mayonesa monodosis	198009.0	2
Alimentación	ALI-0012	Aceite de oliva virgen extra	187787.0	3
Café e infusiones	CAF-0005	Infusión poleo menta Tostadero del Norte	143169.0	1
Café e infusiones	CAF-0017	Sacarina en sobres Café Bahía	128658.0	2
Café e infusiones	CAF-0010	Café en grano mezcla 70/30 Café Bahía	100480.0	3
Cervezas	CER-0066	Ola Fría Sin alcohol	355647.0	1
Cervezas	CER-0020	Brisa Cantábrica Lager	349934.0	2
Cervezas	CER-0060	Ola Fría Tostada	206373.0	3
Desechables	DES-0001	Vasos de cartón 25 cl	385835.0	1
Desechables	DES-0006	Envases take away cartón	297449.0	2
Desechables	DES-0004	Servilletas 40x40 punta-punta	84893.0	3
Licores y destilados	LIC-0006	Brandy Destilerías Cantábricas	240291.0	1
Licores y destilados	LIC-0008	Ginebra Casa Sierra	125200.0	2
Licores y destilados	LIC-0019	Licor de hierbas Montaña Palentina Spiri…	119142.0	3
Limpieza e higiene	LIM-0006	Bobina industrial Higiene Norte	198128.0	1
Limpieza e higiene	LIM-0007	Papel higiénico industrial Higiene Norte	121263.0	2
Limpieza e higiene	LIM-0013	Desengrasante cocina LimpiaPro	57153.0	3
Refrescos	REF-0003	Cola Bahía	421423.0	1
Refrescos	REF-0016	Tónica Brava	291850.0	2
Refrescos	REF-0019	Gaseosa La Pasiega	241763.0	3
Vinos	VIN-0025	Bodegas Valle del Nansa Blanco I.G.P. Co…	917337.0	1
Vinos	VIN-0023	Bodegas Valle del Nansa Crianza D.O. Rib…	586206.0	2
Vinos	VIN-0015	Hacienda Ruesga Reserva D.O.Ca. Rioja	426057.0	3

27 rows — 5 columns

Fíjate en el `QUALIFY puesto <= 3`: en DuckDB se puede usar el alias de la ventana. Nueve categorías, tres productos cada una, 27 filas. El formato está en el `GROUP BY` (dos formatos del mismo producto son productos distintos, como en el MJD-202) aunque no se muestre, para que quepa en la página.

5.3 Quitar duplicados eligiendo cuál se queda

En silver quitaste las líneas duplicadas con `DISTINCT`. Funciona porque las dos copias eran idénticas en todo menos en `linea_id`. Pero ¿y si un día llegan dos versiones de la misma línea con precios distintos, porque alguien la corrigió en el ERP? `DISTINCT` conservaría las dos, porque ya no son iguales, y la venta se contaría dos veces.

La forma robusta es numerar las copias de cada `(pedido_id, num_linea)` y quedarse con una según un criterio explícito. Por ejemplo, la que se insertó primero (el `linea_id` más bajo):

Para contarlas, el primer impulso es poner el `count(*)` en la misma consulta:

SQL · EN DUCKDB

```
SELECT count(*) AS lineas_unicas
FROM bronze.lineas_pedido
QUALIFY row_number() OVER (PARTITION BY pedido_id, num_linea ORDER BY linea_id) = 1;
```

ERROR

```
Binder Error: column "pedido_id" must appear in the GROUP BY clause or must be part of an aggregate
function.
Either add it to the GROUP BY list, or use "ANY_VALUE(pedido_id)" if the exact value of "pedido_id" is
not important.
```

Vuelve a la tabla del 4.3: el `count(*)` es un agregado y se calcula en el paso 3, antes de la ventana (paso 5) y del `QUALIFY` (paso 6). Al haber un agregado sin `GROUP BY`, toda la tabla se convierte en una sola fila, y la ventana ya no tiene `pedido_id` ni `num_linea` con los que partir. Primero se filtra y después se cuenta, cada cosa en su consulta:

SQL · EN DUCKDB

```
SELECT count(*) AS lineas_unicas
FROM (SELECT *
      FROM bronze.lineas_pedido
      QUALIFY row_number() OVER (PARTITION BY pedido_id, num_linea ORDER BY linea_id) = 1);
```

RESULTADO

```
┌────────────────┐
│ lineas_unicas  │
│     int64      │
├────────────────┤
│         380208 │
└────────────────┘
```

380.208, la misma cifra del Volumen 02 por otro camino. La diferencia es que ahora la regla está escrita: "de cada línea, la primera que se insertó". Si mañana el criterio cambia a "la última versión", cambias `ORDER BY linea_id` por `ORDER BY linea_id DESC` y listo. Esto es lo que verás en Databricks con el nombre de **deduplicación**, y es casi siempre un `row_number()`.

5.4 Repartir en grupos iguales: ntile

`ntile(n)` reparte las filas, en orden, en `n` grupos de tamaño parecido. Con `n = 4` son cuartiles; con `n = 5`, quintiles. ¿Cuánto pesan los clientes del primer quintil (el 20 % que más compra) en las ventas del último año?

SQL · EN DUCKDB

```
WITH ventas_cliente AS (
    SELECT p.cliente_maestro_id, sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha > DATE '2026-09-30' - INTERVAL 1 YEAR
      AND p.cliente_maestro_id IS NOT NULL
    GROUP BY p.cliente_maestro_id
),
con_quintil AS (
    SELECT *, ntile(5) OVER (ORDER BY ventas DESC) AS quintil
    FROM ventas_cliente
)
SELECT quintil,
       count(*) AS clientes,
       round(sum(ventas)) AS ventas,
       round(100 * sum(ventas) / sum(sum(ventas)) OVER (), 1) AS pct_ventas
FROM con_quintil
GROUP BY quintil
ORDER BY quintil;
```

RESULTADO

quintil	clientes	ventas	pct_ventas
int64	int64	double	double
1	219	10867657.0	59.6
2	219	3197656.0	17.5
3	218	2015480.0	11.1
4	218	1414015.0	7.8
5	218	725534.0	4.0

El 20 % de los clientes hace casi el 60 % de las ventas. Es una distribución muy habitual (se conoce como **principio de Pareto**, o regla del 80/20), y para un director comercial es de las cifras más importantes que existen: perder a un cliente del primer quintil cuesta lo mismo que perder a quince del último. La retomarás en el ticket MJD-307.

6. Acumulados y medias móviles

6.1 Añadir ORDER BY a la ventana

Hasta ahora, dentro del `OVER` solo había `PARTITION BY`, y cada fila veía su grupo entero. Si añades `ORDER BY`, cambia el significado: cada fila ve **las filas de su grupo hasta ella misma**. Con `sum`, eso es un acumulado:

SQL · EN DUCKDB

```
WITH ventas_mes AS (
    SELECT date_trunc('month', p.fecha)::DATE AS mes,
           sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha >= '2026-01-01'
    GROUP BY mes
)
SELECT mes,
       round(ventas) AS ventas,
       round(sum(ventas) OVER (ORDER BY mes)) AS acumulado
FROM ventas_mes
ORDER BY mes;
```

RESULTADO

mes date	ventas double	acumulado double
2026-01-01	1187705.0	1187705.0
2026-02-01	1066618.0	2254323.0
2026-03-01	1351890.0	3606213.0
2026-04-01	1500290.0	5106503.0
2026-05-01	1582346.0	6688849.0
2026-06-01	1805026.0	8493875.0
2026-07-01	2142228.0	10636103.0
2026-08-01	1975639.0	12611742.0
2026-09-01	1571643.0	14183385.0

Cada mes suma su venta a lo acumulado hasta el mes anterior. Es lo que en una empresa se llama **YTD** (*year to date*, lo que va de año) cuando se reinicia cada enero. Para que se reinicie, se parte por año:

SQL · EN DUCKDB

```
WITH ventas_mes AS (
    SELECT date_trunc('month', p.fecha)::DATE AS mes,
           sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha >= '2025-07-01'
    GROUP BY mes
)
SELECT mes,
       round(ventas) AS ventas,
       round(sum(ventas) OVER (PARTITION BY year(mes) ORDER BY mes)) AS ytd
FROM ventas_mes
ORDER BY mes;
```

RESULTADO

mes date	ventas double	ytd double
2025-07-01	1718695.0	1718695.0
2025-08-01	1673393.0	3392088.0
2025-09-01	1473799.0	4865887.0
2025-10-01	1290332.0	6156219.0
2025-11-01	1120485.0	7276704.0
2025-12-01	1627833.0	8904538.0
2026-01-01	1187705.0	1187705.0
2026-02-01	1066618.0	2254323.0
2026-03-01	1351890.0	3606213.0
2026-04-01	1500290.0	5106503.0
2026-05-01	1582346.0	6688849.0
2026-06-01	1805026.0	8493875.0
2026-07-01	2142228.0	10636103.0
2026-08-01	1975639.0	12611742.0
2026-09-01	1571643.0	14183385.0

15 rows 3 columns

En enero de 2026 el acumulado vuelve a empezar.

6.2 El marco: qué filas ve cada fila

Lo que cada fila "ve" dentro de su partición se llama el **marco** (*frame*). Cuando pones `ORDER BY` sin decir nada más, el marco por defecto es "desde el principio hasta la fila actual". Se puede escribir explícitamente, y así se puede cambiar:

MARCO	QUÉ FILAS VE CADA FILA	PARA QUÉ
`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`	Desde la primera hasta ella	Acumulados
`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`	Las dos anteriores y ella: tres filas	Medias móviles de 3 periodos
`ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`	La anterior, ella y la siguiente	Suavizar una serie
`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`	Todas las de la partición	Lo mismo que sin `ORDER BY`

La media móvil de tres meses quita el ruido de un mes suelto y deja ver la tendencia:

SQL · EN DUCKDB

```
WITH ventas_mes AS (
    SELECT date_trunc('month', p.fecha)::DATE AS mes,
           sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha >= '2026-01-01'
    GROUP BY mes
)
SELECT mes,
       round(ventas) AS ventas,
       round(avg(ventas) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS
media_movil_3m,
       count(*)      OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)  AS meses_en_la_media
FROM ventas_mes
ORDER BY mes;
```

RESULTADO

mes date	ventas double	media_movil_3m double	meses_en_la_media int64
2026-01-01	1187705.0	1187705.0	1
2026-02-01	1066618.0	1127162.0	2
2026-03-01	1351890.0	1202071.0	3
2026-04-01	1500290.0	1306266.0	3
2026-05-01	1582346.0	1478175.0	3
2026-06-01	1805026.0	1629220.0	3
2026-07-01	2142228.0	1843200.0	3
2026-08-01	1975639.0	1974298.0	3
2026-09-01	1571643.0	1896504.0	3

Fíjate en la última columna: enero y febrero no tienen dos meses detrás, así que su "media de tres meses" es en realidad de uno y de dos. Un gráfico que muestre esos dos puntos como si fueran iguales que los demás engaña. Lo habitual es mostrarlos vacíos o explicarlo.

6.3 La trampa de ROWS frente a RANGE

El marco por defecto no es exactamente "desde la primera fila hasta esta", sino `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. La diferencia solo se nota cuando hay **empates en el `ORDER BY`**, y entonces se nota mucho. El acumulado de portes de los primeros pedidos de septiembre, ordenando por fecha:

SQL · EN DUCKDB

```
SELECT fecha, pedido_id, gastos_envio,
       sum(gastos_envio) OVER (ORDER BY fecha) AS con_range,
       sum(gastos_envio) OVER (ORDER BY fecha, pedido_id
                               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS con_rows
FROM silver.pedidos
WHERE fecha >= '2026-09-01' AND estado <> 'Cancelado'
ORDER BY fecha, pedido_id
LIMIT 8;
```

RESULTADO

fecha date	pedido_id int64	gastos_envio decimal(10,2)	con_range decimal(38,2)	con_rows decimal(38,2)
2026-09-01	2026022317	0.00	306.00	0.00
2026-09-01	2026022318	0.00	306.00	0.00
2026-09-01	2026022319	12.00	306.00	12.00
2026-09-01	2026022320	0.00	306.00	12.00
2026-09-01	2026022321	0.00	306.00	12.00
2026-09-01	2026022322	0.00	306.00	12.00
2026-09-01	2026022323	0.00	306.00	12.00
2026-09-01	2026022324	12.00	306.00	24.00

Con `RANGE`, todas las filas del 1 de septiembre tienen el mismo acumulado, 306 €: el de **todo el día**. `RANGE` trata como una sola posición a todas las filas con el mismo valor de ordenación (se llaman *peers*). Con `ROWS`, cada fila suma de una en una. Ninguno está mal, pero son cosas distintas, y si esperabas lo segundo y obtienes lo primero, el gráfico sale en escalones.

IDEA CLAVE

Si el `ORDER BY` de la ventana puede tener empates, decide tú el marco: `ROWS` y un desempate (`ORDER BY fecha, pedido_id`) para ir fila a fila; `RANGE` si de verdad quieres el total del día en cada fila. No dejes que lo decida el valor por defecto.

7. Mirar atrás y adelante

7.1 LAG y LEAD

`lag(columna)` devuelve el valor de esa columna en la **fila anterior** de la ventana; `lead(columna)`, en la siguiente. Es la herramienta para cualquier comparación "con el periodo anterior". La variación de las ventas mes a mes:

SQL · EN DUCKDB

```
WITH ventas_mes AS (
    SELECT date_trunc('month', p.fecha)::DATE AS mes,
           sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha >= '2026-01-01'
    GROUP BY mes
)
SELECT mes,
       round(ventas) AS ventas,
       round(lag(ventas) OVER (ORDER BY mes)) AS mes_anterior,
       round(100 * (ventas / lag(ventas) OVER (ORDER BY mes) - 1), 1) AS variacion_pct
FROM ventas_mes
ORDER BY mes;
```

RESULTADO

mes date	ventas double	mes_anterior double	variacion_pct double
2026-01-01	1187705.0	NULL	NULL
2026-02-01	1066618.0	1187705.0	-10.2
2026-03-01	1351890.0	1066618.0	26.7
2026-04-01	1500290.0	1351890.0	11.0
2026-05-01	1582346.0	1500290.0	5.5
2026-06-01	1805026.0	1582346.0	14.1
2026-07-01	2142228.0	1805026.0	18.7
2026-08-01	1975639.0	2142228.0	-7.8
2026-09-01	1571643.0	1975639.0	-20.4

DESMONTANDO EL COMANDO

`lag(ventas, 1, 0) OVER (PARTITION BY ... ORDER BY mes)`

`lag( ventas`	La columna cuyo valor quieres de otra fila.
`, 1`	Cuántas filas hacia atrás. Por defecto, 1. `lag(ventas, 12)` sería "doce filas atrás".
`, 0 )`	Qué devolver si no hay fila anterior. Por defecto, `NULL`, que suele ser lo correcto: enero no tiene mes anterior dentro de este resultado, y poner un 0 haría pensar que en diciembre no se vendió nada.
`OVER (ORDER BY mes)`	Imprescindible: "anterior" solo tiene sentido con un orden. Con `PARTITION BY`, la fila anterior se busca dentro del grupo.

La variación de agosto a septiembre es una caída del 20 %, y de julio a agosto otra del 8 %. ¿Malas noticias? No: es la estacionalidad del verano que viste en el Volumen 02. Por eso las variaciones mes a mes de un negocio estacional se miran poco; lo que se mira es la comparación con el mismo mes del año anterior (capítulo 8).

7.2 Días entre pedidos

`lag` sirve igual con fechas. ¿Cada cuántos días pide un cliente? Para cada pedido, la diferencia con el anterior **del mismo cliente**:

SQL · EN DUCKDB

```
SELECT cliente_maestro_id,
       fecha,
       lag(fecha) OVER (PARTITION BY cliente_maestro_id ORDER BY fecha_pedido) AS pedido_anterior,
       fecha - lag(fecha) OVER (PARTITION BY cliente_maestro_id ORDER BY fecha_pedido) AS
dias_desde_anterior
FROM silver.pedidos
WHERE estado <> 'Cancelado' AND cliente_maestro_id = 10001
ORDER BY fecha_pedido
LIMIT 8;
```

RESULTADO

cliente_maestro_id	fecha	pedido_anterior	dias_desde_anterior
int64	date	date	int64
10001	2024-12-16	NULL	NULL
10001	2024-12-17	2024-12-16	1
10001	2024-12-26	2024-12-17	9
10001	2025-01-03	2024-12-26	8
10001	2025-01-31	2025-01-03	28
10001	2025-02-05	2025-01-31	5
10001	2025-02-19	2025-02-05	14
10001	2025-03-15	2025-02-19	24

El primer pedido no tiene anterior: `NULL`. A partir de aquí, la media de esa columna por cliente es su intervalo habitual entre pedidos, y comparar el reciente con el histórico detecta a quien empieza a irse antes de que se vaya. Es el ticket MJD-304.

7.3 first_value, last_value y la trampa del marco

`first_value(x)` y `last_value(x)` devuelven el valor de la primera y la última fila del marco. Parece la forma natural de poner, al lado de cada pedido de un cliente, su primer y su último pedido:

SQL · EN DUCKDB

```
SELECT pedido_id, fecha,
       first_value(fecha) OVER (ORDER BY fecha_pedido) AS primero,
       last_value(fecha)  OVER (ORDER BY fecha_pedido) AS ultimo
FROM silver.pedidos
WHERE cliente_maestro_id = 10001 AND estado <> 'Cancelado'
ORDER BY fecha_pedido
LIMIT 5;
```

RESULTADO

pedido_id	fecha	primero	ultimo
int64	date	date	date
2024022796	2024-12-16	2024-12-16	2024-12-16
2024023016	2024-12-17	2024-12-16	2024-12-17
2024023682	2024-12-26	2024-12-16	2024-12-26
2025000172	2025-01-03	2024-12-16	2025-01-03
2025001700	2025-01-31	2024-12-16	2025-01-31

`primero` está bien, pero `ultimo` es siempre la fecha de la propia fila. No es un fallo: es el marco por defecto del 6.2. Con `ORDER BY`, cada fila ve "desde el principio hasta ella", así que la última fila de su marco es **ella misma**. Para que vea toda la partición hay que decirlo:

SQL · EN DUCKDB

```
SELECT pedido_id, fecha,
       first_value(fecha) OVER w AS primero,
       last_value(fecha)  OVER w AS ultimo
FROM silver.pedidos
WHERE cliente_maestro_id = 10001 AND estado <> 'Cancelado'
WINDOW w AS (ORDER BY fecha_pedido ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
ORDER BY fecha_pedido
LIMIT 5;
```

RESULTADO

pedido_id	fecha	primero	ultimo
int64	date	date	date
2024022796	2024-12-16	2024-12-16	2026-09-25
2024023016	2024-12-17	2024-12-16	2026-09-25
2024023682	2024-12-26	2024-12-16	2026-09-25
2025000172	2025-01-03	2024-12-16	2026-09-25
2025001700	2025-01-31	2024-12-16	2026-09-25

MÁS FÁCIL CON MIN Y MAX

Para fechas, `min(fecha) OVER (PARTITION BY ...)` y `max(fecha) OVER (PARTITION BY ...)`, sin `ORDER BY`, dan lo mismo sin preocuparse del marco. `first_value` y `last_value` merecen la pena cuando lo que quieres no es la fecha, sino **otra columna** de la primera o la última fila: el canal del primer pedido, por ejemplo.

7.4 El último pedido de cada cliente, cuatro formas

Es la pregunta más repetida en las entrevistas de SQL, en sus mil variantes (el último pedido, el salario más alto de cada departamento, el producto más caro de cada categoría). Lo difícil no es la fecha, que sale con un `max`, sino **el resto de la fila**: el importe, el canal o el comercial de ese último pedido.

Forma 1: agrupar y volver a unir. La clásica, la que funciona en cualquier SQL antiguo:

SQL · EN DUCKDB

```
WITH ultimo AS (
    SELECT cliente_maestro_id, max(fecha_pedido) AS fecha_pedido
    FROM silver.pedidos
    WHERE estado <> 'Cancelado' AND cliente_maestro_id IS NOT NULL
    GROUP BY cliente_maestro_id
)
SELECT count(*) AS filas, count(DISTINCT p.cliente_maestro_id) AS clientes
FROM silver.pedidos AS p
JOIN ultimo AS u USING (cliente_maestro_id, fecha_pedido)
WHERE p.estado <> 'Cancelado';
```

RESULTADO

```
┌────────┬──────────┐
│ filas  │ clientes │
│ int64  │  int64   │
├────────┼──────────┤
│  1175  │     1175 │
└────────┴──────────┘
```

Una fila por cliente, pero por suerte. El `JOIN` vuelve a buscar, para cada cliente, los pedidos con su fecha máxima. Si un cliente tuviera dos pedidos registrados en el mismo minuto (por ejemplo, uno por teléfono y otro por la web), el `JOIN` encontraría los dos y ese cliente saldría duplicado. En estos datos no ocurre, pero nada lo impide. Es el punto débil de esta forma, y por eso la comprobación `filas = clientes` debería acompañarla siempre.

Forma 2: `row_number()` y `QUALIFY`. Una fila exacta por cliente, con un desempate explícito:

SQL · EN DUCKDB

```
SELECT count(*) AS filas, count(DISTINCT cliente_maestro_id) AS clientes
FROM (SELECT *
      FROM silver.pedidos
      WHERE estado <> 'Cancelado' AND cliente_maestro_id IS NOT NULL
      QUALIFY row_number() OVER (PARTITION BY cliente_maestro_id
                                 ORDER BY fecha_pedido DESC, pedido_id DESC) = 1);
```

RESULTADO

```
┌────────┬──────────┐
│ filas  │ clientes │
│ int64  │  int64   │
├────────┼──────────┤
│  1175  │     1175 │
└────────┴──────────┘
```

Una fila por cliente. El `count` va fuera, por lo que viste en el 5.3.

Forma 3: `arg_max`. Una función de DuckDB (también existe en Databricks como `max_by`) que responde exactamente a la pregunta "¿cuál es el valor de esta columna en la fila donde esta otra es máxima?":

SQL · EN DUCKDB

```
SELECT cliente_maestro_id,
       max(fecha_pedido)                AS ultimo_pedido,
       arg_max(pedido_id, fecha_pedido)  AS pedido_id,
       arg_max(canal, fecha_pedido)      AS canal
FROM silver.pedidos
WHERE estado <> 'Cancelado' AND cliente_maestro_id IS NOT NULL
GROUP BY cliente_maestro_id
ORDER BY cliente_maestro_id
LIMIT 5;
```

RESULTADO

cliente_maestro_id int64	ultimo_pedido timestamp	pedido_id int64	canal varchar
10001	2026-09-25 11:34:00	2026024707	Teléfono
10002	2026-09-10 17:21:00	2026023331	Comercial
10003	2026-07-30 19:06:00	2026018609	Web B2B
10004	2026-09-07 19:12:00	2026022975	Web B2B
10005	2026-09-30 11:24:00	2026025114	Comercial

`arg_max(pedido_id, fecha_pedido)` se lee "el `pedido_id` de la fila con la `fecha_pedido` máxima". Es la más corta, pero no existe en SQL Server.

Forma 4: la subconsulta correlacionada del 2.5. Funciona, pero la medirás en el capítulo 9 y verás por qué no se recomienda.

FORMA	VENTAJA	INCONVENIENTE	DÓNDE FUNCIONA
Agrupar y volver a unir	Funciona en cualquier SQL	Duplica con empates	En todas partes
`row_number()` + filtro	Una fila exacta; desempate explícito	Más larga sin `QUALIFY`	En todas partes (con CTE en SQL Server)
`arg_max` / `max_by`	La más corta y clara	No es estándar	DuckDB, Databricks
Subconsulta correlacionada	Fácil de leer	Lenta en muchos motores	En todas partes

PARA LAS ENTREVISTAS

Si te piden "el último X de cada Y" en una pizarra, escribe la forma 2 con CTE y `WHERE rn = 1`: funciona en cualquier motor, demuestra que conoces las funciones de ventana y te permite hablar del desempate. Después menciona `QUALIFY` y `arg_max` como atajos. Es la respuesta de alguien con experiencia.

PARTE 3

Fechas avanzadas

Calendarios completos, meses sin ventas y comparaciones con el año anterior que no se engañan solas.

8. Calendarios y comparaciones

8. Calendarios y comparaciones

8.1 Una fila por día o por mes, aunque no haya datos

Una consulta de ventas por mes solo devuelve los meses en los que hubo ventas. Parece lógico, pero es una trampa: un mes sin ventas **no aparece**, y lo que no aparece no se ve. En un gráfico, la línea une el mes anterior con el siguiente como si no hubiera pasado nada; en una comparación con el mes anterior, el "anterior" pasa a ser el de hace dos meses.

La solución es fabricar primero todas las fechas que deberían existir y unir después los datos. DuckDB lo hace con `generate_series`:

SQL · EN DUCKDB

```
SELECT mes::DATE AS mes
FROM generate_series(DATE '2026-01-01', DATE '2026-09-01', INTERVAL 1 MONTH) AS t(mes);
```

RESULTADO

```
┌────────────┐
│    mes     │
│    date    │
├────────────┤
│ 2026-01-01 │
│ 2026-02-01 │
│ 2026-03-01 │
│ 2026-04-01 │
│ 2026-05-01 │
│ 2026-06-01 │
│ 2026-07-01 │
│ 2026-08-01 │
│ 2026-09-01 │
└────────────┘
```

DESMONTANDO EL COMANDO

`generate_series(inicio, fin, paso) AS t(mes)`

`generate_series( ... )`	Una función que devuelve una tabla: una fila por cada valor desde el inicio hasta el fin, ambos incluidos.
`DATE '2026-01-01'`	El primer valor.
`DATE '2026-09-01'`	El último. Incluido: `generate_series` es inclusiva. Su hermana `range` no incluye el final.
`INTERVAL 1 MONTH`	El salto entre una fila y la siguiente. Puede ser `1 DAY`, `7 DAY`, `1 YEAR`...
`AS t(mes)`	Nombre de la tabla y de su columna.
`mes::DATE`	Con fechas, `generate_series` devuelve `TIMESTAMP` (fecha con hora 00:00). Se convierte a `DATE` para unir con las fechas de silver.

8.2 La dimensión calendario

Una lista de fechas sola no aporta mucho; lo útil es añadirle todo lo que se pregunta sobre una fecha: año, trimestre, mes, nombre del mes, día de la semana, si es fin de semana. Esa tabla se llama **calendario** o **dimensión fecha**, y existe en todos los modelos analíticos. En el Volumen 04 la guardarás en gold; aquí la construyes por primera vez:

SQL · EN DUCKDB

```
SELECT d::DATE                                  AS fecha,
       year(d)                                   AS anio,
       quarter(d)                                AS trimestre,
       month(d)                                  AS mes,
       strftime(d, '%Y-%m')                      AS anio_mes,
       isodow(d)                                 AS dia_semana,
       isodow(d) >= 6                            AS es_fin_de_semana,
       date_trunc('month', d)::DATE              AS inicio_mes
FROM generate_series(DATE '2024-01-01', DATE '2026-09-30', INTERVAL 1 DAY) AS t(d)
ORDER BY fecha;
```

RESULTADO

fecha date	anio int64	trimestre int64	mes int64	anio_mes varchar	dia_semana int64	es_fin_de_semana boolean	inicio_mes date
2024-01-01	2024	1	1	2024-01	1	false	2024-01-01
2024-01-02	2024	1	1	2024-01	2	false	2024-01-01
2024-01-03	2024	1	1	2024-01	3	false	2024-01-01
2024-01-04	2024	1	1	2024-01	4	false	2024-01-01
·	·	·	·	·	·	·	·
·	·	·	·	·	·	·	·
·	·	·	·	·	·	·	·
2026-09-27	2026	3	9	2026-09	7	true	2026-09-01
2026-09-28	2026	3	9	2026-09	1	false	2026-09-01
2026-09-29	2026	3	9	2026-09	2	false	2026-09-01
2026-09-30	2026	3	9	2026-09	3	false	2026-09-01

1004 rows (8 shown) 8 columns

1.004 días, uno por fila, del 1 de enero de 2024 al 30 de septiembre de 2026. Con esta tabla, preguntas como "ventas por día de la semana" o "comparar los mismos días laborables" pasan a ser un `JOIN` y un `GROUP BY`.

¿Cuántos pedidos entran cada día de la semana?

SQL · EN DUCKDB

```
SELECT isodow(fecha) AS dia_semana,
       CASE isodow(fecha)
           WHEN 1 THEN 'lunes'     WHEN 2 THEN 'martes'  WHEN 3 THEN 'miércoles'
           WHEN 4 THEN 'jueves'    WHEN 5 THEN 'viernes' WHEN 6 THEN 'sábado'
           WHEN 7 THEN 'domingo'
       END AS dia,
       count(*) AS pedidos
FROM silver.pedidos
GROUP BY ALL
ORDER BY dia_semana;
```

RESULTADO

dia_semana int64	dia varchar	pedidos int64
1	lunes	16621
2	martes	14467
3	miércoles	13954
4	jueves	14560
5	viernes	13524
6	sábado	4322

El lunes es el día fuerte (los bares reponen lo vendido el fin de semana), el sábado entra muy poco y el domingo no aparece: no es que haya cero pedidos, es que **no hay fila**. Exactamente la trampa de 8.1. Si Marcos te pidiera los pedidos por día de la semana para planificar turnos, el domingo debería salir con un 0 explícito, no desaparecer.

8.3 Meses sin ventas

El patrón completo para que los huecos aparezcan: **todas las combinaciones que deberían existir** (con un `CROSS JOIN`), unidas con un `LEFT JOIN` a los datos, y los vacíos convertidos en cero. Pedidos por provincia y mes, contando los meses en que una provincia no tuvo ninguno:

SQL · EN DUCKDB

```
WITH meses AS (
    SELECT mes::DATE AS mes
    FROM generate_series(DATE '2024-01-01', DATE '2026-09-01', INTERVAL 1 MONTH) AS t(mes)
),
provincias AS (
    SELECT DISTINCT provincia FROM silver.clientes
),
pedidos_mes AS (
    SELECT date_trunc('month', p.fecha)::DATE AS mes, c.provincia, count(*) AS pedidos
    FROM silver.pedidos AS p
    JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
    WHERE p.estado <> 'Cancelado'
    GROUP BY ALL
)
SELECT pr.provincia,
       count(*) AS meses,
       count(*) FILTER (WHERE coalesce(pm.pedidos, 0) = 0) AS meses_sin_pedidos,
       min(m.mes) FILTER (WHERE pm.pedidos > 0) AS primer_mes_con_pedidos
FROM meses AS m
CROSS JOIN provincias AS pr
LEFT JOIN pedidos_mes AS pm ON pm.mes = m.mes AND pm.provincia = pr.provincia
GROUP BY pr.provincia
ORDER BY pr.provincia;
```

RESULTADO

provincia	meses	meses_sin_pedidos	primer_mes_con_pedidos
varchar	int64	int64	date
Asturias	33	0	2024-01-01
Bizkaia	33	14	2025-03-01
Burgos	33	0	2024-01-01
Cantabria	33	0	2024-01-01
Palencia	33	0	2024-01-01

33 meses por cinco provincias: 165 combinaciones. Bizkaia tiene 14 meses vacíos, de enero de 2024 a febrero de 2025, porque Marejada entró allí en marzo de 2025. Sin el calendario, esos catorce meses no existirían en el resultado y nadie los echaría de menos. Con el calendario, aparecen y se pueden explicar.

DESMONTANDO EL COMANDO

`meses CROSS JOIN provincias LEFT JOIN pedidos_mes ON mes Y provincia`

`meses CROSS JOIN provincias`	Todas las parejas posibles de mes y provincia: el "esqueleto" de la tabla que quieres, con o sin datos.
`LEFT JOIN pedidos_mes`	Pega los datos donde los haya. Donde no, deja `NULL`.
`ON pm.mes = m.mes AND pm.provincia = pr.provincia`	Se une por **las dos** columnas del esqueleto. Si te olvidas una, el abanico del Volumen 02 vuelve a aparecer.
`coalesce(pm.pedidos, 0)`	Un mes sin datos es un mes con cero pedidos. Aquí el `NULL` sí significa cero, y se dice explícitamente.

8.4 La trampa de comparar con la fila anterior

En el 7.1 calculaste la variación con el mes anterior con `lag()`. `lag()` no sabe nada de meses: devuelve la **fila** anterior. Si la serie tiene huecos, la fila anterior no es el mes anterior. Mira una cafetería de Ribamontán al Mar, que en invierno pide muy poco:

SQL · EN DUCKDB

```
WITH pedidos_mes AS (
    SELECT date_trunc('month', fecha)::DATE AS mes, count(*) AS pedidos
    FROM silver.pedidos
    WHERE cliente_maestro_id = 11147 AND estado <> 'Cancelado'
      AND fecha >= '2025-01-01'
    GROUP BY mes
)
SELECT mes, pedidos,
       lag(mes)     OVER (ORDER BY mes) AS fila_anterior_es_de,
       lag(pedidos) OVER (ORDER BY mes) AS pedidos_fila_anterior
FROM pedidos_mes
ORDER BY mes;
```

RESULTADO

```
┌────────────┬──────────┬──────────────────────┬──────────────────────┐
│    mes     │ pedidos  │ fila_anterior_es_de  │ pedidos_fila_anterior │
│    date    │  int64   │         date         │         int64         │
├────────────┼──────────┼──────────────────────┼──────────────────────┤
│ 2025-01-01 │        2 │ NULL                 │                 NULL │
│ 2025-03-01 │        1 │ 2025-01-01           │                    2 │
│ 2025-04-01 │        3 │ 2025-03-01           │                    1 │
│ 2025-05-01 │        5 │ 2025-04-01           │                    3 │
│ 2025-06-01 │        1 │ 2025-05-01           │                    5 │
│ 2025-09-01 │        1 │ 2025-06-01           │                    1 │
│ 2025-12-01 │        2 │ 2025-09-01           │                    1 │
│ 2026-03-01 │        1 │ 2025-12-01           │                    2 │
│ 2026-04-01 │        1 │ 2026-03-01           │                    1 │
│ 2026-05-01 │        1 │ 2026-04-01           │                    1 │
│ 2026-07-01 │        1 │ 2026-05-01           │                    1 │
│ 2026-08-01 │        3 │ 2026-07-01           │                    1 │
│ 2026-09-01 │        3 │ 2026-08-01           │                    3 │
└────────────┴──────────┴──────────────────────┴──────────────────────┘
  13 rows                                                  4 columns
```

La columna `fila_anterior_es_de` lo delata: en varios meses, la fila "anterior" es de hace dos, tres o más meses, porque en medio no hubo pedidos y esas filas no existen. Una variación "frente al mes anterior" calculada así compararía, por ejemplo, un mes de primavera con uno de otoño del año anterior. Y no da ningún error.

La solución es la de 8.3: primero el calendario completo, después la ventana.

SQL · EN DUCKDB

```
WITH meses AS (
    SELECT mes::DATE AS mes
    FROM generate_series(DATE '2025-01-01', DATE '2026-09-01', INTERVAL 1 MONTH) AS t(mes)
),
pedidos_mes AS (
    SELECT date_trunc('month', fecha)::DATE AS mes, count(*) AS pedidos
    FROM silver.pedidos
    WHERE cliente_maestro_id = 11147 AND estado <> 'Cancelado'
    GROUP BY mes
),
completo AS (
    SELECT m.mes, coalesce(pm.pedidos, 0) AS pedidos
    FROM meses AS m
    LEFT JOIN pedidos_mes AS pm USING (mes)
)
SELECT mes, pedidos,
       lag(pedidos) OVER (ORDER BY mes)     AS mes_anterior,
       lag(pedidos, 12) OVER (ORDER BY mes) AS mismo_mes_anio_anterior
FROM completo
ORDER BY mes;
```

RESULTADO

mes date	pedidos int64	mes_anterior int64	mismo_mes_anio_anterior int64
2025-01-01	2	NULL	NULL
2025-02-01	0	2	NULL
2025-03-01	1	0	NULL
2025-04-01	3	1	NULL
2025-05-01	5	3	NULL
2025-06-01	1	5	NULL
2025-07-01	0	1	NULL
.	.	.	.
.	.	.	.
.	.	.	.
2026-03-01	1	0	1
2026-04-01	1	1	3
2026-05-01	1	1	5
2026-06-01	0	1	1
2026-07-01	1	0	0
2026-08-01	3	1	0
2026-09-01	3	3	1

21 rows (14 shown) 4 columns

Ahora cada fila es un mes, haya pedidos o no, y `lag(pedidos, 12)` es de verdad "el mismo mes del año anterior". Los primeros doce meses no tienen año anterior en este resultado: `NULL`, como debe ser.

IDEA CLAVE

`lag()` mira la fila anterior, no el periodo anterior. Antes de una comparación temporal, asegúrate de que hay exactamente una fila por periodo: calendario completo y `LEFT JOIN`. Es el error más frecuente en los informes de variaciones, y la primera pregunta de un buen revisor.

8.5 Interanual: comparar con el mismo mes del año anterior

En un negocio estacional, la comparación que importa es la **interanual** (en inglés *YoY*, *year over year*): septiembre de 2026 contra septiembre de 2025. Hay dos formas de escribirla. Con `lag(..., 12)` sobre una serie completa, como acabas de ver. O con un `JOIN` de la tabla consigo misma, desplazando la fecha un año, que no depende de que la serie esté completa:

SQL · EN DUCKDB

```
WITH ventas_mes AS (
    SELECT date_trunc('month', p.fecha)::DATE AS mes,
           sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
    GROUP BY mes
)
SELECT a.mes,
       round(a.ventas) AS ventas,
       round(b.ventas) AS ventas_anio_anterior,
       round(100 * (a.ventas / b.ventas - 1), 1) AS variacion_interanual_pct
FROM ventas_mes AS a
LEFT JOIN ventas_mes AS b ON b.mes = a.mes - INTERVAL 1 YEAR
WHERE a.mes >= '2026-01-01'
ORDER BY a.mes;
```

RESULTADO

mes date	ventas double	ventas_anio_anterior double	variacion_interanual_pct double
2026-01-01	1187705.0	974274.0	21.9
2026-02-01	1066618.0	848370.0	25.7
2026-03-01	1351890.0	1063694.0	27.1
2026-04-01	1500290.0	1218471.0	23.1
2026-05-01	1582346.0	1256817.0	25.9
2026-06-01	1805026.0	1353213.0	33.4
2026-07-01	2142228.0	1718695.0	24.6
2026-08-01	1975639.0	1673393.0	18.1
2026-09-01	1571643.0	1473799.0	6.6

Todos los meses de 2026 crecen frente a 2025, entre un 18 % y un 33 %, salvo septiembre, que solo crece un 6,6 %. Antes de alarmar a nadie, el MJD-303 te pedirá averiguar por qué.

DESMONTANDO LA CONSULTA

`LEFT JOIN ventas_mes AS b ON b.mes = a.mes - INTERVAL 1 YEAR`

`ventas_mes AS a`	Este año: la fila que se muestra.
`ventas_mes AS b`	La misma CTE con otro alias, en el papel de "el año anterior".
`b.mes = a.mes - INTERVAL 1 YEAR`	Cada mes se une con el que tiene su misma fecha un año antes. No depende del orden ni de que falten meses.
`LEFT JOIN`	Para que un mes sin año anterior salga con `NULL` en lugar de desaparecer.

8.6 Acumulado del año contra acumulado del año anterior

La otra comparación favorita de cualquier dirección: "¿cómo vamos en lo que va de año frente al año pasado a estas alturas?". El truco es comparar periodos **equivalentes**: de enero a septiembre de 2026 contra de enero a septiembre de 2025, no contra el año 2025 entero.

SQL · EN DUCKDB

```
SELECT year(p.fecha) AS anio,
       round(sum(l.importe_neto)) AS ventas_ene_sep
FROM silver.pedidos AS p
JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
WHERE p.estado <> 'Cancelado'
  AND month(p.fecha) <= 9
GROUP BY anio
ORDER BY anio;
```

RESULTADO

```
┌────────┬──────────────────┐
│ anio   │ ventas_ene_sep   │
│ int64  │     double       │
├────────┼──────────────────┤
│  2024  │      9653614.0   │
│  2025  │     11580726.0   │
│  2026  │     14183385.0   │
└────────┴──────────────────┘
```

De enero a septiembre, 2026 va un 22,5 % por encima de 2025. Es una cifra justa porque compara nueve meses con nueve meses. Compararla con el total de 2025 (15,6 millones) diría que "vamos por debajo del año pasado", que es falso: faltan octubre, noviembre y diciembre.

EL ÚLTIMO MES PUEDE ESTAR A MEDIAS

Septiembre de 2026 está completo porque los datos llegan hasta el día 30. Pero si la fecha de corte fuera el 15 de septiembre, el "septiembre" de 2026 tendría medio mes, y compararlo con el septiembre completo de 2025 daría una caída que no existe. En los informes reales esto pasa cada mes: o se compara hasta el mismo día del año anterior, o se excluye el mes en curso.

PARTE 4

Rendimiento y dialectos

Por qué una consulta tarda, cómo leer lo que el motor piensa hacer, qué cambios merecen la pena y cómo se escribe todo esto en Fabric y Databricks.

9. Rendimiento

10. El mismo SQL en otros sitios

9. Rendimiento

9.1 Por qué importa, aunque aquí todo vaya rápido

Con 77.000 pedidos, casi todo tarda milésimas en tu Mac. Pero las mismas consultas, en Fabric o en Databricks, correrán sobre cientos de millones de filas, y ahí una consulta mal escrita puede tardar horas y **costar dinero**: en la nube se paga por el cómputo que se gasta. Además, los exámenes DP-600 y Databricks preguntan explícitamente por rendimiento. Aquí vas a aprender a mirar, que es lo que no cambia de un motor a otro.

9.2 Medir: .timer

Lo activaste en el 1.2. Si no lo tienes, `.timer on`. Cada consulta muestra al final su tiempo:

RESULTADO

```
┌─────────────┬─────────────┐
│    canal    │ pedidos     │
│   varchar   │   int64     │
├─────────────┼─────────────┤
│ Comercial   │       13768 │
│ Teléfono    │        5852 │
│ Web B2B     │        5546 │
└─────────────┴─────────────┘
Run Time (s): real 0.002 user 0.002793 sys 0.001685
```

`real` es el tiempo de reloj, el que esperas tú. `user` suma el tiempo de todos los núcleos del procesador (por eso puede ser mayor que `real`, como viste con `time` en el MJD-101). Para comparar dos versiones de una consulta, ejecuta cada una dos o tres veces y quédate con la más baja: la primera ejecución a veces paga cargar datos del disco.

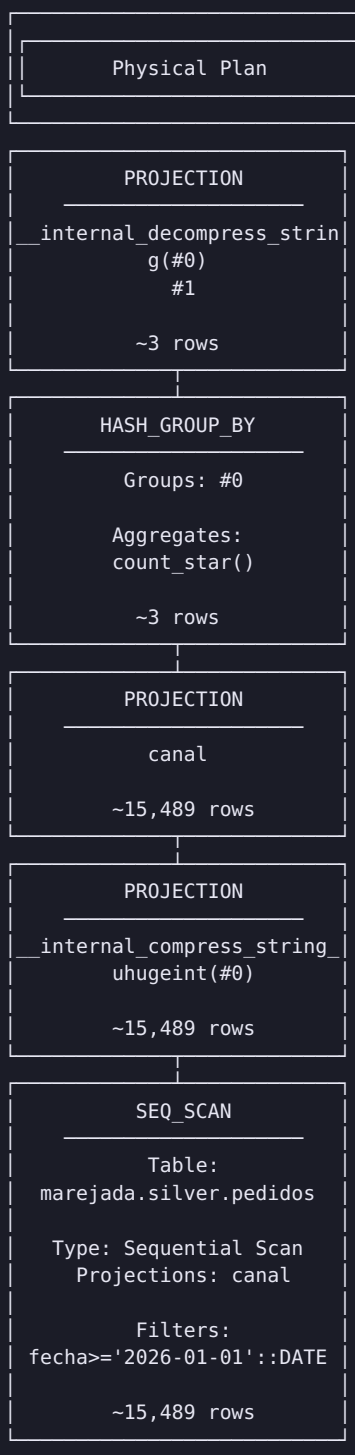
9.3 EXPLAIN: lo que el motor piensa hacer

Antes de ejecutar una consulta, el motor decide **cómo** ejecutarla: qué tabla leer primero, en qué orden unir, dónde filtrar. Esa decisión se llama **plan de ejecución**, y `EXPLAIN` lo muestra sin ejecutar nada:

TÚ ESCRIBES

```
EXPLAIN
SELECT canal, count(*) AS pedidos
FROM silver.pedidos
WHERE fecha >= '2026-01-01'
GROUP BY canal;
```

RESULTADO



Un plan se lee **de abajo arriba**: los datos entran por abajo y suben, caja a caja, hasta el resultado. Cada caja es un **operador**:

OPERADOR	QUÉ HACE	LO QUE TE DICE AQUÍ
`SEQ_SCAN`	Lee la tabla	Que lee `silver.pedidos`, **solo la columna** `canal` (*Projections*) y que el filtro de la fecha se aplica **mientras lee** (*Filters*)
`PROJECTION`	Calcula o elige columnas	Pasos internos de DuckDB para comprimir y descomprimir textos. No los escribiste tú y puedes ignorarlos
`HASH_GROUP_BY`	Agrupa	Agrupa por la columna `#0` (el canal) y cuenta
`~15,489 rows`	Filas que **estima** que pasarán por ahí	Es una estimación hecha antes de ejecutar (9.4)

Dos cosas que el plan confirma y que son la base del rendimiento en todas las herramientas analíticas del máster:

1. **Solo lee las columnas que usas.** La tabla tiene 13 columnas y el plan lee una. Es el almacenamiento por columnas del Volumen 01 (4.2) funcionando. Por eso `SELECT *` en una tabla grande es caro, y en Fabric o Databricks, literalmente, dinero.
2. **Filtra al leer.** El filtro de la fecha no espera a tener toda la tabla en memoria: se aplica en el `SEQ_SCAN`. A eso se le llama **empujar el filtro** (*filter pushdown*), y es la razón por la que filtrar pronto, en el `WHERE`, es casi siempre lo más rápido.

9.4 EXPLAIN ANALYZE: lo que ha hecho de verdad

`EXPLAIN ANALYZE` ejecuta la consulta y muestra el mismo plan con lo que ha pasado realmente: filas reales y tiempo de cada operador. Aquí, el principio y la caja del escaneo (los tiempos de tu Mac serán parecidos, nunca idénticos):

TÚ ESCRIBES

```
EXPLAIN ANALYZE
SELECT canal, count(*) AS pedidos
FROM silver.pedidos
WHERE fecha >= '2026-01-01'
GROUP BY canal;
```

RESULTADO

```
┌─────────────────────────────────────┐
│┌───────────────────────────────────┐│
│└───────────────────────────────────┘│
│             Total Time: 0.0053s     │
└─────────────────────────────────────┘
   ... (las cajas intermedias, recortadas) ...
┌─────────────┴─────────────┐
│         TABLE_SCAN        │
│    ─────────────────────  │
│           Table:          │
│  marejada.silver.pedidos  │
│                           │
│   Type: Sequential Scan   │
│     Projections: canal    │
│                           │
│          Filters:         │
│ fecha>='2026-01-01'::DATE │
│                           │
│        25,166 rows        │
│           0.00s           │
└───────────────────────────┘
```

El escaneo estimaba unas 15.489 filas y han pasado 25.166. Las estimaciones son aproximadas, y en consultas sencillas da igual. Pero cuando una consulta lenta tiene un operador cuya estimación se equivoca por mucho, ahí suele estar el problema: el motor ha elegido el plan pensando que pasarían pocas filas, y han pasado muchas.

DÓNDE MIRAR EN UN PLAN LENTO

Busca el operador con más tiempo y, debajo de él, el primero en el que el número de filas se dispara. Las tres causas más comunes: un `JOIN` sin condición o con la condición equivocada (multiplica filas: el abanico), un filtro que no se ha empujado hasta el escaneo y una subconsulta correlacionada que el motor no ha sabido convertir en `JOIN`.

9.5 Una consulta lenta y su arreglo

Para cada pedido, ¿cuántos pedidos había hecho el mismo cliente en los 90 días anteriores? Es una medida de lo "caliente" que está un cliente. Con una subconsulta correlacionada se escribe tal cual se piensa:

TÚ ESCRIBES

```
SELECT count(*) AS pedidos, sum(previos_90d) AS suma_control
FROM (
    SELECT p.pedido_id,
           (SELECT count(*)
            FROM silver.pedidos AS q
            WHERE q.cliente_maestro_id = p.cliente_maestro_id
              AND q.estado <> 'Cancelado'
              AND q.fecha_pedido <  p.fecha_pedido
              AND q.fecha_pedido >= p.fecha_pedido - INTERVAL 90 DAY) AS previos_90d
    FROM silver.pedidos AS p
    WHERE p.estado <> 'Cancelado' AND p.cliente_maestro_id IS NOT NULL
);
```

RESULTADO

```
┌──────────┬─────────────────┐
│ pedidos  │ suma_control    │
│  int64   │    int128       │
├──────────┼─────────────────┤
│   74497  │       609030    │
└──────────┴─────────────────┘
Run Time (s): real 0.060 user 0.080613 sys 0.006918
```

Y con una función de ventana, con un marco de tipo `RANGE` que, en lugar de contar filas, mide una distancia en el tiempo:

TÚ ESCRIBES

```
SELECT count(*) AS pedidos, sum(previos_90d) AS suma_control
FROM (
    SELECT pedido_id,
           count(*) OVER (PARTITION BY cliente_maestro_id
                          ORDER BY fecha_pedido
                          RANGE BETWEEN INTERVAL 90 DAY PRECEDING AND CURRENT ROW) - 1 AS previos_90d
    FROM silver.pedidos
    WHERE estado <> 'Cancelado' AND cliente_maestro_id IS NOT NULL
);
```

RESULTADO

```
┌──────────┬─────────────────┐
│ pedidos  │ suma_control    │
│  int64   │    int128       │
├──────────┼─────────────────┤
│    74497 │          609030 │
└──────────┴─────────────────┘
Run Time (s): real 0.006 user 0.023589 sys 0.002335
```

Las dos dan el mismo resultado, y la suma de control lo confirma. Medidas en un MacBook Air M2 con DuckDB 1.5.6, la de ventana es unas diez veces más rápida: 0,006 s frente a 0,060 s. La correlacionada, conceptualmente, busca para cada uno de los 74.497 pedidos los pedidos de su cliente en una ventana de tiempo: DuckDB la reescribe internamente como un `JOIN`, pero aun así compara cada pedido con todos los de su cliente. La de ventana ordena los pedidos de cada cliente una vez y recorre la lista con el marco deslizante.

DESMONTANDO LA CONSULTA

`count(*) OVER (PARTITION BY cliente ORDER BY fecha_pedido RANGE BETWEEN INTERVAL 90 DAY PRECEDING AND CURRENT ROW) - 1`

`PARTITION BY cliente_maestro_id`	Cada cliente, por separado.
`ORDER BY fecha_pedido`	Sus pedidos, en orden.
`RANGE BETWEEN INTERVAL 90 DAY PRECEDING`	El marco empieza en el pedido que esté 90 días antes que el actual, **en tiempo**, no en número de filas. Con `ROWS`, "90 PRECEDING" serían 90 pedidos.
`AND CURRENT ROW`	Y termina en el actual, incluido.
`- 1`	El marco incluye el propio pedido; se resta para contar solo los anteriores. (Si dos pedidos del mismo cliente tuvieran exactamente la misma hora, `RANGE` incluiría los dos como *peers*: en estos datos no pasa, y la suma de las dos consultas coincide.)

En DuckDB la diferencia es de milisegundos. En un motor que no sepa reescribir la correlacionada, o con cien millones de pedidos, es la diferencia entre segundos y horas.

9.6 Lo que de verdad hace rápida una consulta analítica

COSTUMBRE	POR QUÉ
Selecciona solo las columnas que necesitas	Los motores por columnas solo leen lo que pides (9.3)

COSTUMBRE	POR QUÉ
Filtra lo antes posible, en el `WHERE`	El filtro se empuja al escaneo y todo lo de arriba trabaja con menos filas
Agrega antes de unir	El MJD-204 del Volumen 02: unir 380.000 líneas con 77.000 pedidos es más caro que unir 77.000 importes por pedido
Funciones de ventana en lugar de autojoins y correlacionadas	Una pasada ordenada en lugar de comparaciones fila contra fila (9.5)
No envuelvas en funciones la columna que filtras	`WHERE year(fecha) = 2026` obliga a calcular el año de cada fila antes de poder descartarla; `WHERE fecha >= '2026-01-01' AND fecha < '2027-01-01'` deja al motor usar las estadísticas de la columna y saltarse bloques enteros

¿Y LOS ÍNDICES?

En ASIR viste los índices: la estructura que acelera las búsquedas en MySQL o SQL Server. En las bases analíticas se usan muy poco. DuckDB guarda automáticamente el mínimo y el máximo de cada columna por bloques de filas (*zonemaps*) y con eso se salta los bloques que no cumplen el filtro; los índices que admite solo compensan para buscar filas sueltas. En Fabric y Databricks verás otras técnicas con el mismo objetivo (particiones, *V-Order*, *liquid clustering*), y todas se basan en lo mismo: no leer lo que no hace falta.

10. El mismo SQL en otros sitios

Las funciones de ventana y las CTE son SQL estándar y funcionan casi igual en todas partes. Las diferencias están en los atajos y en las fechas:

QUÉ	DUCKDB	SQL SERVER / FABRIC WAREHOUSE	DATABRICKS (SPARK SQL)
Filtrar por una ventana	`QUALIFY rn = 1`	No existe: CTE y `WHERE rn = 1`	`QUALIFY rn = 1`
Valor de otra columna en el máximo	`arg_max(x, f)`	No existe: `row_number()` y filtro	`max_by(x, f)`
CTE recursiva	`WITH RECURSIVE`	`WITH` (sin la palabra `RECURSIVE`)	`WITH RECURSIVE` (versiones recientes)
Serie de fechas	`generate_series(a, b, INTERVAL 1 DAY)`	`GENERATE_SERIES` solo con números en muchas versiones; se suman días a una fecha	`sequence(a, b, interval 1 day)` con `explode`
Mediana	`median(x)`	`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x) OVER (...)`	`median(x)` o `percentile(x, 0.5)`
Restar un año	`f - INTERVAL 1 YEAR`	`DATEADD(year, -1, f)`	`add_months(f, -12)` o `f - INTERVAL 1 YEAR`
Días entre dos fechas	`f2 - f1`	`DATEDIFF(day, f1, f2)`	`datediff(f2, f1)`
Ventana con nombre	`WINDOW w AS (...)`	`WINDOW` (desde SQL Server 2022)	`WINDOW w AS (...)`
Marco por tiempo	`RANGE BETWEEN INTERVAL 90 DAY PRECEDING ...`	Solo `ROWS` o `RANGE` sin intervalos	`RANGE BETWEEN INTERVAL 90 DAYS PRECEDING ...`

El patrón más importante de la tabla es el primero, porque lo vas a escribir en el DP-600. El último pedido de cada cliente en el Warehouse de Fabric:

```sql
WITH numerados AS (
    SELECT pedido_id, cliente_maestro_id, fecha_pedido, canal,
           ROW_NUMBER() OVER (PARTITION BY cliente_maestro_id
                              ORDER BY fecha_pedido DESC, pedido_id DESC) AS rn
    FROM silver.pedidos
    WHERE estado <> 'Cancelado'
)
SELECT pedido_id, cliente_maestro_id, fecha_pedido, canal
FROM numerados
WHERE rn = 1;
```

PARA EL DP-600 Y EL DE DATABRICKS

Los dos exámenes preguntan funciones de ventana en contexto: elegir entre `ROW_NUMBER`, `RANK` y `DENSE_RANK` según cómo deben tratarse los empates, deduplicar quedándose con la fila más reciente, o calcular un acumulado. Y el de Databricks pregunta `QUALIFY` y `max_by`. Si entiendes el 5.1 y el 7.4, tienes esas preguntas.

TICKETS

Otra semana en Marejada

Nueve encargos que necesitan lo que acabas de aprender. Guarda cada consulta en sql/03-sql-ii/ e intenta cada uno antes de mirar la solución.

Tickets · Volumen 03

Soluciones · Volumen 03

Tickets · Volumen 03

Fecha de corte: **30 de septiembre de 2026**. Salvo que el ticket diga otra cosa, ventas con el criterio del 1.4: líneas de pedidos no cancelados, por fecha de pedido y por cliente maestro.

TICKET MJD-301 De: Paula Herrero · Marketing y tienda online

Prioridad: media

Hola, Álvaro. Lo que me propuso Elena después del MJD-208: la web ya es casi el 28 % de las ventas, pero no sé si nos está trayendo clientes nuevos o si son los de siempre, que antes pedían por teléfono. Para el plan de marketing de 2027 necesito saberlo.

CRITERIOS DE ACEPTACIÓN

- Cuántos clientes han comprado alguna vez por la web, y de ellos cuántos eran nuevos (su primer pedido fue ya por la web) y cuántos ya compraban por otro canal.
- De los que ya compraban, por qué canal lo hacían sobre todo antes de pasarse a la web.
- Qué parte de las ventas web viene de cada grupo.

TICKET MJD-302 De: Íñigo Sainz · Director comercial

Prioridad: alta Plazo: viernes

Para el comité: los cinco mejores clientes de cada provincia en los últimos doce meses, con lo que pesa cada uno dentro de su provincia. Si alguno se nos va, quiero saber cuánto nos duele.

CRITERIOS DE ACEPTACIÓN

- Cinco filas por provincia, con el puesto, el nombre del cliente, sus ventas y su porcentaje sobre las ventas de la provincia.
- Clientes reales: las fichas duplicadas, sumadas.

TICKET MJD-303 De: Carmen Ruiz · Directora general

Prioridad: alta

He visto en el informe que en septiembre solo hemos crecido un 6,6 % frente al año pasado, cuando todo el año vamos por encima del 20 %. ¿Ha pasado algo? Una frase y la cifra que lo demuestre.

TICKET MJD-304 De: Raquel Fernández · Jefa de ventas Cantabria

Prioridad: alta

Lo de los clientes dormidos (MJD-203) llegó tarde: cuando un cliente lleva 90 días sin pedir, ya se ha ido. Quiero enterarme antes. ¿Puedes sacarme los clientes de Cantabria que están pidiendo mucho menos de lo que suelen?

CRITERIOS DE ACEPTACIÓN

- Que la alerta dependa de cada cliente: no es lo mismo 20 días sin pedir para un bar que pide cada semana que para uno que pide cada mes.
- Sin los que ya llevan 90 días o más sin pedir: esos ya los tengo.

PISTA

Para cada cliente, calcula cada cuántos días suele pedir y compáralo con los días que lleva sin hacerlo. Pregúntate si para "lo que suele" te sirve más la media o la mediana.

TICKET MJD-305 De: Lucía Gómez · Administración y finanzas

Prioridad: media

Necesito para el cierre la facturación de 2026, mes a mes y por provincia, con el acumulado del año y la comparación con el mismo acumulado de 2025. Con nuestros criterios de siempre (los del MJD-204).

CRITERIOS DE ACEPTACIÓN

- Todas las provincias en todos los meses, aunque no haya facturación.
- Acumulado de 2026 y de 2025 hasta el mismo mes, y la variación.

TICKET MJD-306 De: Marcos Diego · Logística y almacén

Prioridad: media

Con el inventario del 30 de septiembre, ¿para cuántas semanas tengo stock de cada producto al ritmo al que se está vendiendo? Quiero ver lo que se me va a acabar y lo que me sobra.

CRITERIOS DE ACEPTACIÓN

- Por producto activo: unidades en stock (los dos almacenes juntos), venta media semanal de las últimas ocho semanas completas y semanas de cobertura.
- Ojo con las semanas en las que un producto no se ha vendido: cuentan como cero.

TICKET MJD-307 De: Íñigo Sainz · Director comercial

Prioridad: media

Siempre se dice que el 20 % de los clientes hace el 80 % de las ventas. ¿Es verdad en Marejada? ¿Cuántos clientes necesito para llegar al 80 % de lo vendido en los últimos doce meses?

TICKET MJD-308 De: Elena Pérez · Responsable de datos

Prioridad: media

Íñigo quiere una "ficha" de cada cliente para el CRM nuevo. Una fila por cliente real, con su primer y su último pedido, el canal y el comercial de ese último pedido, sus pedidos y ventas de los últimos doce meses y su puesto dentro de su provincia.

ENTREGABLE

La consulta en `sql/03-sql-ii/` y el resultado en `entregables/mjd-308_ficha_clientes.xlsx`.

TICKET MJD-309 De: Elena Pérez · Responsable de datos

Prioridad: baja

He encontrado una consulta en una carpeta del analista que estaba antes que tú (te la copio debajo). Calcula el puesto de cada cliente en su provincia por sus ventas de 2026, y dicen que tarda mucho. Arréglala, demuéstrame que da lo mismo y dime cuánto has ganado.

SQL

```
WITH ventas AS (
    SELECT p.cliente_maestro_id, c.provincia, sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
    WHERE p.estado <> 'Cancelado' AND year(p.fecha) >= 2026
    GROUP BY ALL
)
SELECT v.*,
       (SELECT count(*) FROM ventas AS w
        WHERE w.provincia = v.provincia AND w.ventas > v.ventas) + 1 AS puesto
FROM ventas AS v;
```

Soluciones · Volumen 03

MJD-301 · ¿Los clientes de la web son nuevos?

El razonamiento, en pasos que se pueden comprobar uno a uno (por eso CTE):

1. Para cada cliente que ha usado la web, la fecha de su **primer** pedido web.
2. Sus pedidos **anteriores** a esa fecha, por canal.
3. Si no tiene ninguno, es nuevo. Si tiene, gana el canal con más pedidos.

SQL · EN DUCKDB

```
WITH validos AS (
    SELECT * FROM silver.pedidos
    WHERE estado <> 'Cancelado' AND cliente_maestro_id IS NOT NULL
),
primer_web AS (
    SELECT cliente_maestro_id, min(fecha_pedido) AS primer_pedido_web
    FROM validos
    WHERE canal = 'Web B2B'
    GROUP BY cliente_maestro_id
),
historial_previo AS (
    SELECT w.cliente_maestro_id,
           count(v.pedido_id)                                    AS pedidos_antes,
           count(v.pedido_id) FILTER (WHERE v.canal = 'Teléfono')  AS por_telefono,
           count(v.pedido_id) FILTER (WHERE v.canal = 'Comercial') AS por_comercial
    FROM primer_web AS w
    LEFT JOIN validos AS v
           ON v.cliente_maestro_id = w.cliente_maestro_id
          AND v.fecha_pedido < w.primer_pedido_web
    GROUP BY w.cliente_maestro_id
),
clasificados AS (
    SELECT cliente_maestro_id,
           CASE WHEN pedidos_antes = 0            THEN '1. Nuevo'
                WHEN por_telefono > por_comercial THEN '2. Venía del teléfono'
                ELSE                                   '3. Venía del comercial'
           END AS origen
    FROM historial_previo
),
ventas_web AS (
    SELECT v.cliente_maestro_id, sum(l.importe_neto) AS ventas
    FROM validos AS v
    JOIN silver.lineas_pedido AS l ON l.pedido_id = v.pedido_id
    WHERE v.canal = 'Web B2B'
    GROUP BY v.cliente_maestro_id
)
SELECT c.origen,
       count(*) AS clientes,
       round(100.0 * count(*) / sum(count(*)) OVER (), 1) AS pct_clientes,
       round(sum(w.ventas)) AS ventas_web,
       round(100 * sum(w.ventas) / sum(sum(w.ventas)) OVER (), 1) AS pct_ventas_web
FROM clasificados AS c
JOIN ventas_web AS w USING (cliente_maestro_id)
GROUP BY c.origen
ORDER BY c.origen;
```

RESULTADO

origen varchar	clientes int64	pct_clientes double	ventas_web double	pct_ventas_web double
1. Nuevo	49	4.5	240896.0	5.5
2. Venía del teléfono	37	3.4	97297.0	2.2
3. Venía del comercial	1000	92.1	4068986.0	92.3

DESMONTANDO LA CONSULTA

```
LEFT JOIN validos AS v ON v.cliente_maestro_id = w.cliente_maestro_id AND v.fecha_pedido < w.primer_pedido_web
```

`LEFT JOIN`	Para no perder a los clientes sin historial previo: precisamente son los nuevos. Con `INNER JOIN` desaparecerían y la primera fila del resultado no existiría.
`AND v.fecha_pedido < w.primer_pedido_web`	La condición de fecha va en el `ON`, no en el `WHERE`. En el `WHERE`, eliminaría las filas con `NULL` del `LEFT JOIN` y lo convertiría en un `INNER JOIN` sin avisar.
`count(v.pedido_id)`	Cuenta pedidos previos, no filas: para un cliente nuevo, la única fila tiene `NULL` en `v.pedido_id` y el recuento da 0. Con `count(*)` daría 1.

Un cliente es "nuevo" si no tiene ningún pedido anterior al primero por la web. Si ya compraba, cuenta como "del teléfono" cuando más de la mitad de sus pedidos previos fueron por teléfono; en caso contrario (incluido el empate), como "del comercial".

La respuesta a Paula: de los 1.086 clientes que han comprado por la web, solo 49 (un 4,5 %, que hacen el 5,5 % de las ventas web) eran nuevos. El 95 % ya eran clientes, y casi todos (1.000) venían sobre todo de las visitas comerciales; del teléfono, solo 37. La web no está captando clientes: está cambiando cómo piden los que ya teníamos, y sobre todo sustituye la visita del comercial.

REVISIÓN DE ELENA

Este es de los buenos, por dos razones. La primera, técnica: la condición de fecha en el `ON` y el `count(v.pedido_id)`. Son exactamente los dos detalles que hacen que esta consulta funcione, y los dos fallan sin dar error si se escriben de la otra forma. Si en una entrevista te preguntan la diferencia entre poner una condición en el `ON` o en el `WHERE` de un `LEFT JOIN`, este es tu ejemplo.

La segunda, de negocio, y es la que le importa a Paula: el resultado contradice lo que ella esperaba. Dilo con tacto y con la cifra delante, y añade lo que sigue: si la web sustituye a la visita del comercial, el comercial tiene tiempo libre para buscar clientes nuevos, que es justo lo que la web no hace. Esa frase convierte un dato en una recomendación.

Un matiz para que lo sepas tú: "nuevo" aquí es "su primer pedido en nuestros datos fue por la web". Un cliente que compraba en 2023 y volvió en 2025 por la web saldría como nuevo, porque los datos empiezan en 2024. Con 49 casos no cambia la conclusión, pero ponlo en una nota.

MJD-302 · Los cinco mejores clientes de cada provincia

Ventas por cliente maestro en los últimos doce meses, su provincia, un `rank()` dentro de cada provincia y el peso con una ventana sobre la provincia:

SQL · EN DUCKDB

```sql
WITH ventas_cliente AS (
    SELECT c.provincia, c.cliente_id, c.nombre_comercial,
           sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha > DATE '2026-09-30' - INTERVAL 1 YEAR
    GROUP BY c.provincia, c.cliente_id, c.nombre_comercial
)
SELECT provincia,
       rank() OVER (PARTITION BY provincia ORDER BY ventas DESC) AS puesto,
       nombre_comercial,
       round(ventas) AS ventas,
       round(100 * ventas / sum(ventas) OVER (PARTITION BY provincia), 1) AS pct_provincia
FROM ventas_cliente
QUALIFY puesto <= 5
ORDER BY provincia, puesto;
```

RESULTADO

provincia varchar	puesto int64	nombre_comercial varchar	ventas double	pct_provincia double
Asturias	1	Supermercado La Sidra	388548.0	11.7
Asturias	2	Hotel La Judería	195750.0	5.9
Asturias	3	Supermercado La Rula	157043.0	4.7
Asturias	4	Supermercado El Mercado	138296.0	4.2
Asturias	5	Supermercado Bitácora	115784.0	3.5
Bizkaia	1	Hotel Albatros	143195.0	7.3
Bizkaia	2	Supermercado La Montaña	98620.0	5.0
Bizkaia	3	Supermercado Los Arcos	81705.0	4.2
Bizkaia	4	Supermercado El Cid	81232.0	4.2
Bizkaia	5	Hotel La Muralla	71659.0	3.7
Burgos	1	Hotel Mareas	130249.0	7.8
Burgos	2	Supermercado Arena	123428.0	7.4
Burgos	3	Supermercado Bitácora	115910.0	6.9
Burgos	4	Supermercado La Montaña	88471.0	5.3
Burgos	5	Supermercado La Taberna	73039.0	4.4
Cantabria	1	Supermercado Los Arcos	168611.0	1.6
Cantabria	2	Supermercado Los Pinares	163458.0	1.6
Cantabria	3	Supermercado Velero	128407.0	1.2
Cantabria	4	Supermercado El Ancla	116815.0	1.1
Cantabria	5	Supermercado Nautilus	115321.0	1.1
Palencia	1	Supermercado Duna	133309.0	14.3
Palencia	2	Supermercado La Ría	86077.0	9.2
Palencia	3	Hotel El Abra	62956.0	6.7
Palencia	4	Hotel Los Pinares II	51791.0	5.6
Palencia	5	Hotel La Estación	46575.0	5.0

25 rows 5 columns

Fíjate en que hay dos ventanas distintas en la misma consulta: `rank()` con `ORDER BY` y la suma de la provincia sin él. Y en que el porcentaje se calcula **antes** del `QUALIFY`: la suma de la provincia incluye a todos sus clientes, no solo a los cinco que se muestran. Si filtraras primero (en una CTE) y calcularas el porcentaje después, cada provincia sumaría el 100 % entre sus cinco primeros.

REVISIÓN DE ELENA

Correcto, y bien unido: con `JOIN silver.clientes ON c.cliente_id = p.cliente_maestro_id` la provincia y el nombre son los de la ficha maestra, así que las fichas duplicadas suman en su cliente real.

Lo que le añadiría para Íñigo es la lectura, porque una tabla de 25 filas no se lee en un comité. Fuera de Cantabria, el primer cliente de cada provincia pesa entre el 7 % y el 14 % de sus ventas (en Palencia, el 14,3 %); en Cantabria, con muchos más clientes, ninguno pasa del 1,6 %. Perder al primero de Palencia es perder uno de cada siete euros de la provincia. Esa es la frase. Y una pregunta que te harán: ¿por qué `rank()` y no `row_number()`? Porque si dos clientes empataran en ventas, los dos merecen estar en la lista.

MJD-303 · ¿Qué ha pasado en septiembre?

Lo primero es no dar por buena la cifra sin mirar el contexto. Las ventas de julio, agosto y septiembre de los tres años, con la variación frente al año anterior y frente a hace **dos** años:

SQL · EN DUCKDB

```
WITH ventas_mes AS (
    SELECT date_trunc('month', p.fecha)::DATE AS mes, sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
    GROUP BY mes
)
SELECT a.mes,
       round(a.ventas) AS ventas,
       round(100 * (a.ventas / b.ventas - 1), 1) AS vs_anio_anterior_pct,
       round(100 * (a.ventas / c.ventas - 1), 1) AS vs_hace_dos_anios_pct
FROM ventas_mes AS a
LEFT JOIN ventas_mes AS b ON b.mes = a.mes - INTERVAL 1 YEAR
LEFT JOIN ventas_mes AS c ON c.mes = a.mes - INTERVAL 2 YEAR
WHERE month(a.mes) IN (7, 8, 9)
ORDER BY a.mes;
```

RESULTADO

mes date	ventas double	vs_anio_anterior_pct double	vs_hace_dos_anios_pct double
2024-07-01	1436498.0	NULL	NULL
2024-08-01	1389560.0	NULL	NULL
2024-09-01	1085462.0	NULL	NULL
2025-07-01	1718695.0	19.6	NULL
2025-08-01	1673393.0	20.4	NULL
2025-09-01	1473799.0	35.8	NULL
2026-07-01	2142228.0	24.6	49.1
2026-08-01	1975639.0	18.1	42.2
2026-09-01	1571643.0	6.6	44.8

La clave está en la fila de septiembre de 2025: aquel mes creció un 35,8 %, muchísimo más que julio y agosto. Septiembre de 2026 se compara con un septiembre de 2025 excepcionalmente bueno. Si se compara con 2024, septiembre de 2026 crece un 44,8 %, en la línea de julio (49,1 %) y agosto (42,2 %).

La respuesta a Carmen: "No ha pasado nada en septiembre de 2026: es que septiembre de 2025 fue excepcional (+35,8 %). Frente a 2024, septiembre crece un 44,8 %, igual que julio y agosto." A ese fenómeno se le llama **efecto base**: una variación depende tanto del periodo actual como del periodo con el que se compara.

REVISIÓN DE ELENA

Muy bien: has contestado con una frase y una cifra, que es lo que pidió Carmen, y has buscado la explicación en la base de la comparación antes que en el mes actual. Es el primer sitio donde hay que mirar siempre que una variación se sale de lo normal.

Lo que haría yo después, antes de cerrar el tema: comprobar que septiembre de 2026 está completo. Los datos llegan al día 30, sí, pero hay 272 pedidos de los últimos días pendientes o en reparto, que cuentan como venta con nuestro criterio comercial y no con el de contabilidad. Para Carmen no cambia nada, pero si Lucía saca la cifra con el suyo, te dará otro porcentaje y te lo preguntará.

MJD-304 · Clientes que empiezan a irse

La pista apuntaba bien: un intervalo "habitual" debe depender de cada cliente. Para cada cliente de Cantabria:

1. Los días entre cada pedido y el anterior (`lag`, 7.2).
2. Su intervalo habitual: la **mediana** de esos días.
3. Los días que lleva sin pedir a la fecha de corte.
4. Alerta si lleva más de tres veces su intervalo habitual, con un mínimo de tres semanas (para no avisar de un bar que pide cada dos días y lleva siete) y menos de 90 días (los dormidos ya los tiene Raquel).

¿Por qué la mediana y no la media? Porque un solo parón largo (un cliente que cerró un mes por obras) dispara la media y esconde lo que el cliente hace normalmente. La mediana es el valor del medio: la mitad de los intervalos son menores y la otra mitad mayores, y un valor extremo no la mueve.

SQL · EN DUCKDB

```
WITH pedidos_cantabria AS (
    SELECT p.cliente_maestro_id, p.fecha,
           p.fecha - lag(p.fecha) OVER (PARTITION BY p.cliente_maestro_id
                                        ORDER BY p.fecha_pedido) AS dias_desde_anterior
    FROM silver.pedidos AS p
    JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
    WHERE p.estado <> 'Cancelado' AND c.provincia = 'Cantabria'
),
por_cliente AS (
    SELECT cliente_maestro_id,
           count(*)                          AS pedidos,
           median(dias_desde_anterior)       AS intervalo_habitual,
           round(avg(dias_desde_anterior), 1) AS intervalo_medio,
           max(fecha)                        AS ultimo_pedido,
           DATE '2026-09-30' - max(fecha)    AS dias_sin_pedir
    FROM pedidos_cantabria
    GROUP BY cliente_maestro_id
)
SELECT c.nombre_comercial, co.nombre_completo AS comercial,
       pc.intervalo_habitual, pc.dias_sin_pedir,
       round(pc.dias_sin_pedir / pc.intervalo_habitual, 1) AS veces_lo_habitual
FROM por_cliente AS pc
JOIN silver.clientes AS c ON c.cliente_id = pc.cliente_maestro_id
LEFT JOIN silver.comerciales AS co ON co.comercial_id = c.comercial_id
WHERE pc.pedidos >= 10
  AND pc.dias_sin_pedir >= 21
  AND pc.dias_sin_pedir < 90
  AND pc.dias_sin_pedir > 3 * pc.intervalo_habitual
ORDER BY veces_lo_habitual DESC;
```

RESULTADO

nombre_comercial varchar	comercial varchar	…	dias_sin_pedir int64	veces_lo_habitual double
Cafetería La Ría	Daniel Gutiérrez Pérez	…	70	11.7
Restaurante Bahía	Sara Ceballos Ortiz	…	89	11.1
Alimentación El Sardinero	Pablo Revuelta Gómez	…	83	8.3
Alimentación El Muelle	Pablo Revuelta Gómez	…	61	7.6
Sidrería La Sidra	Sara Ceballos Ortiz	…	42	7.0
Alimentación Salitre	Sara Ceballos Ortiz	…	72	6.5
·	·	…	·	·
·	·	…	·	·
·	·	…	·	·
Bar El Mirador	Daniel Gutiérrez Pérez	…	23	3.3
Bar Nautilus	Daniel Gutiérrez Pérez	…	27	3.2
Bar El Ancla	Pablo Revuelta Gómez	…	50	3.1
Sidrería La Peña	Daniel Gutiérrez Pérez	…	28	3.1
Restaurante El Hórreo	Celia Gómez Sáiz	…	28	3.1
Cafetería El Faro	Daniel Gutiérrez Pérez	…	34	3.1

53 rows (12 shown) 5 columns (4 shown)

La lista completa son 53 clientes, ordenados por lo alarmante: el primero suele pedir cada 6 días y lleva 70 sin hacerlo, casi doce veces lo normal. El filtro `pedidos >= 10` evita sacar conclusiones de clientes con muy pocos pedidos, en los que "lo habitual" no se puede calcular con fiabilidad.

REVISIÓN DE ELENA

Esto ya no es un listado: es un sistema de alerta temprana, y de los que funcionan en las empresas de verdad. Tres cosas que haría yo antes de dárselo a Raquel:

- **Explica la regla en una línea encima de la tabla.** "Clientes que llevan más de tres veces su intervalo habitual sin pedir, con un mínimo de tres semanas." Si Raquel no entiende por qué sale un cliente, no lo llamará.
- **Los números 3, 21 y 10 son decisiones tuyas**, no verdades. Díselo y ofrécele moverlos: con "dos veces" saldrían muchos más.
- **Cuidado con la estacionalidad.** Un bar de playa que en invierno pide la mitad saltará en octubre sin que pase nada. Con más tiempo, se compararía con su intervalo habitual **en esa época del año**. Para septiembre, vale.

MJD-305 · Facturación de 2026 por provincia y mes, con acumulados

Tres herramientas del volumen juntas: el calendario completo (8.3) para que estén todas las combinaciones, el criterio contable del MJD-204 (mes de entrega, con portes, agregando cada cosa en su grano) y dos ventanas para los acumulados.

SQL · EN DUCKDB

```
WITH meses AS (
    SELECT mes::DATE AS mes
    FROM generate_series(DATE '2025-01-01', DATE '2026-09-01', INTERVAL 1 MONTH) AS t(mes)
),
provincias AS (
    SELECT DISTINCT provincia FROM silver.clientes
),
importe_pedido AS (
    SELECT pedido_id, sum(importe_neto) AS importe
    FROM silver.lineas_pedido
    GROUP BY pedido_id
),
facturado AS (
    SELECT date_trunc('month', p.fecha_entrega)::DATE AS mes,
           c.provincia,
           sum(i.importe + p.gastos_envio) AS base
    FROM silver.pedidos AS p
    JOIN importe_pedido AS i ON i.pedido_id = p.pedido_id
    JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
    WHERE p.estado IN ('Entregado', 'Devuelto parcial')
    GROUP BY ALL
),
completo AS (
    SELECT m.mes, pr.provincia, coalesce(f.base, 0) AS base
    FROM meses AS m
    CROSS JOIN provincias AS pr
    LEFT JOIN facturado AS f ON f.mes = m.mes AND f.provincia = pr.provincia
),
con_acumulado AS (
    SELECT *, sum(base) OVER (PARTITION BY provincia, year(mes) ORDER BY mes) AS ytd
    FROM completo
)
SELECT a.provincia,
       strftime(a.mes, '%m') AS mes,
       round(a.base) AS base_2026,
       round(a.ytd)  AS ytd_2026,
       round(b.ytd)  AS ytd_2025,
       round(100 * (a.ytd / nullif(b.ytd, 0) - 1), 1) AS var_ytd_pct
FROM con_acumulado AS a
JOIN con_acumulado AS b ON b.provincia = a.provincia AND b.mes = a.mes - INTERVAL 1 YEAR
WHERE year(a.mes) = 2026
ORDER BY a.provincia, a.mes;
```

RESULTADO

provincia	mes	base_2026	ytd_2026	ytd_2025	var_ytd_pct
varchar	varchar	double	double	double	double
Asturias	01	226200.0	226200.0	211163.0	7.1
Asturias	02	193609.0	419810.0	366517.0	14.5
Asturias	03	226127.0	645936.0	571972.0	12.9
Asturias	04	256419.0	902355.0	815051.0	10.7
Asturias	05	294637.0	1196993.0	1054489.0	13.5
Asturias	06	336274.0	1533267.0	1313782.0	16.7
Asturias	07	371597.0	1904864.0	1615520.0	17.9
Asturias	08	365511.0	2270375.0	1930311.0	17.6
Asturias	09	260047.0	2530422.0	2232471.0	13.3
Bizkaia	01	118797.0	118797.0	0.0	NULL
Bizkaia	02	103588.0	222385.0	0.0	NULL
Bizkaia	03	170887.0	393273.0	9993.0	3835.5
Bizkaia	04	180938.0	574211.0	25850.0	2121.3
Bizkaia	05	168618.0	742830.0	50886.0	1359.8
Bizkaia	06	200700.0	943529.0	103562.0	811.1
Bizkaia	07	281328.0	1224858.0	180517.0	578.5
Bizkaia	08	230017.0	1454874.0	265472.0	448.0
Bizkaia	09	182139.0	1637014.0	363215.0	350.7
Burgos	01	117953.0	117953.0	89967.0	31.1
Burgos	02	100136.0	218090.0	173418.0	25.8
Burgos	03	107622.0	325712.0	289981.0	12.3
Burgos	04	145517.0	471228.0	408653.0	15.3
Burgos	05	144101.0	615329.0	521388.0	18.0
Burgos	06	183124.0	798453.0	665171.0	20.0
Burgos	07	191616.0	990069.0	837818.0	18.2
Burgos	08	151400.0	1141469.0	959779.0	18.9
Burgos	09	135112.0	1276581.0	1090514.0	17.1
Cantabria	01	673131.0	673131.0	627227.0	7.3
Cantabria	02	606810.0	1279941.0	1184288.0	8.1
Cantabria	03	783960.0	2063901.0	1862877.0	10.8
Cantabria	04	837401.0	2901302.0	2598479.0	11.7
Cantabria	05	851064.0	3752366.0	3360036.0	11.7
Cantabria	06	1029150.0	4781516.0	4231660.0	13.0
Cantabria	07	1180804.0	5962320.0	5252176.0	13.5
Cantabria	08	1164444.0	7126765.0	6284827.0	13.4
Cantabria	09	862838.0	7989602.0	7196697.0	11.0
Palencia	01	60525.0	60525.0	57685.0	4.9
Palencia	02	69202.0	129727.0	113182.0	14.6
Palencia	03	77827.0	207554.0	178520.0	16.3
Palencia	04	70948.0	278502.0	241728.0	15.2
Palencia	05	79750.0	358252.0	331350.0	8.1
Palencia	06	101525.0	459777.0	409821.0	12.2
Palencia	07	99420.0	559197.0	500914.0	11.6
Palencia	08	103899.0	663096.0	596483.0	11.2
Palencia	09	80188.0	743284.0	689157.0	7.9

45 rows 6 columns

DESMONTANDO LA CONSULTA

`sum(base) OVER (PARTITION BY provincia, year(mes) ORDER BY mes) AS ytd`

`PARTITION BY provincia, year(mes)`	Un acumulado independiente para cada provincia y cada año: en enero vuelve a empezar.
`ORDER BY mes`	Suma desde enero hasta el mes de la fila. Aquí no hay empates (una fila por provincia y mes gracias al calendario), así que `RANGE` y `ROWS` darían lo mismo.
`JOIN con_acumulado AS b ON b.mes = a.mes - INTERVAL 1 YEAR`	El acumulado de 2025 hasta el mismo mes: compara enero-marzo con enero-marzo.
`nullif(b.ytd, 0)`	Bizkaia no facturó nada en enero y febrero de 2025: dividir entre cero daría error o infinito. Con `nullif`, la variación sale `NULL`, que es lo honesto: no se puede calcular un crecimiento desde cero.

En Bizkaia, las variaciones son enormes (un 350 % en septiembre) porque 2025 fue el año de apertura: comparar con un año de arranque dice poco, y conviene decírselo a Lucía. Y Bizkaia muestra justo por qué hacía falta el calendario: sin él, enero y febrero de 2025 no existirían, el `JOIN` con el año anterior no encontraría pareja para enero y febrero de 2026 y esas filas desaparecerían del informe de Lucía.

REVISIÓN DE ELENA

Correcto, y con el `nullif` donde tenía que estar. Dos comentarios de cierre contable, que es lo que Lucía mirará:

- **Los pedidos de clientes borrados del maestro no están.** El `JOIN` con `silver.clientes` los excluye, porque no sabemos su provincia (eran de Cantabria, lo descubriste en el MJD-204, pero silver no lo sabe todavía). Para un informe de cierre, o se pide a Lucía que confirme la provincia de esos seis clientes y se añade a silver, o se muestra una fila "Sin asignar". Lo que no se hace es dejarlos fuera en silencio: dile la cifra.
- **Las 23 entregas imposibles tampoco están**, porque en silver tienen la fecha vacía. Contabilidad sí las factura, en el mes de su fecha errónea. Es la nota que ya le debes desde el MJD-204.

MJD-306 · Semanas de cobertura

Tres piezas: el stock por producto (sumando los dos almacenes), la venta semanal de las ocho últimas semanas **completas** con las semanas sin venta a cero (calendario de semanas), y la división.

Las ocho semanas completas anteriores al corte son las del 3 de agosto al 27 de septiembre: la semana del 28 de septiembre solo tiene tres días de datos y bajaría la media. `date_trunc('week', fecha)` lleva cada fecha al lunes de su semana.

SQL · EN DUCKDB

```
WITH semanas AS (
    SELECT s::DATE AS semana
    FROM generate_series(DATE '2026-08-03', DATE '2026-09-21', INTERVAL 7 DAY) AS t(s)
),
productos_activos AS (
    SELECT producto_id, sku, nombre, formato FROM silver.productos WHERE activo
),
venta_semanal AS (
    SELECT date_trunc('week', p.fecha)::DATE AS semana, l.producto_id, sum(l.cantidad) AS unidades
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha >= '2026-08-03' AND p.fecha < '2026-09-28'
    GROUP BY ALL
),
media AS (
    SELECT pa.producto_id, avg(coalesce(v.unidades, 0)) AS media_semanal
    FROM productos_activos AS pa
    CROSS JOIN semanas AS s
    LEFT JOIN venta_semanal AS v ON v.producto_id = pa.producto_id AND v.semana = s.semana
    GROUP BY pa.producto_id
),
stock AS (
    SELECT producto_id, sum(unidades) AS unidades_stock
    FROM silver.inventario
    WHERE producto_id IS NOT NULL
    GROUP BY producto_id
)
SELECT pa.sku, pa.nombre,
       coalesce(st.unidades_stock, 0) AS stock,
       round(m.media_semanal, 1) AS venta_semanal,
       round(coalesce(st.unidades_stock, 0) / nullif(m.media_semanal, 0), 1) AS semanas_cobertura
FROM productos_activos AS pa
JOIN media AS m USING (producto_id)
LEFT JOIN stock AS st USING (producto_id)
ORDER BY semanas_cobertura
LIMIT 12;
```

RESULTADO

sku varchar	nombre varchar	stock int128	venta_semanal double	semanas_cobertura double
LIC-0021	Ron Old Pier	0	43.4	0.0
REF-0020	Gaseosa La Pasiega	0	69.5	0.0
LIM-0006	Bobina industrial Higiene Norte	0	384.6	0.0
CER-0014	Faro Norte Sin alcohol	0	4.0	0.0
LIC-0012	Licor de hierbas Casa Sierra	0	28.9	0.0
ALI-0012	Aceite de oliva virgen extra	6	151.0	0.0
LIM-0007	Papel higiénico industrial Higiene Norte	0	203.5	0.0
CER-0005	Faro Norte Especial	0	12.3	0.0
REF-0005	Cola Bahía Zero	0	83.6	0.0
CER-0023	Brisa Cantábrica Especial	0	3.3	0.0
LIC-0019	Licor de hierbas Montaña Palentina Spir…	0	224.3	0.0
LIM-0013	Desengrasante cocina LimpiaPro	0	117.6	0.0

12 rows 5 columns

Y el resumen para Marcos, con la misma lógica en tramos:

SQL · EN DUCKDB

```
WITH semanas AS (
    SELECT s::DATE AS semana
    FROM generate_series(DATE '2026-08-03', DATE '2026-09-21', INTERVAL 7 DAY) AS t(s)
),
venta_semanal AS (
    SELECT date_trunc('week', p.fecha)::DATE AS semana, l.producto_id, sum(l.cantidad) AS unidades
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado' AND p.fecha >= '2026-08-03' AND p.fecha < '2026-09-28'
    GROUP BY ALL
),
cobertura AS (
    SELECT pa.producto_id,
           coalesce(any_value(st.unidades_stock), 0) / nullif(avg(coalesce(v.unidades, 0)), 0) AS
semanas
    FROM silver.productos AS pa
    CROSS JOIN semanas AS s
    LEFT JOIN venta_semanal AS v ON v.producto_id = pa.producto_id AND v.semana = s.semana
    LEFT JOIN (SELECT producto_id, sum(unidades) AS unidades_stock
               FROM silver.inventario WHERE producto_id IS NOT NULL
               GROUP BY producto_id) AS st ON st.producto_id = pa.producto_id
    WHERE pa.activo
    GROUP BY pa.producto_id
)
SELECT CASE WHEN semanas < 1   THEN '1. Menos de 1 semana'
            WHEN semanas < 4   THEN '2. De 1 a 4 semanas'
            WHEN semanas <= 26 THEN '3. De 4 a 26 semanas'
            ELSE                    '4. Más de 26 semanas'
       END AS cobertura,
       count(*) AS productos
FROM cobertura
GROUP BY ALL
ORDER BY cobertura;
```

RESULTADO

cobertura varchar	productos int64
1. Menos de 1 semana	72
2. De 1 a 4 semanas	80
3. De 4 a 26 semanas	52
4. Más de 26 semanas	7

Los doce primeros de la lista no tienen ni una unidad en ninguno de los dos almacenes: son roturas de stock ya, no riesgos.

La respuesta a Marcos: de los 211 productos activos, 72 tienen stock para menos de una semana al ritmo actual (12 de ellos, con el stock a cero) y 7 para más de medio año.

REVISIÓN DE ELENA

Bien resuelto, y con la trampa de las semanas sin venta evitada: si hubieras calculado la media solo sobre las semanas con ventas, un producto que se vende una semana de cada cuatro parecería venderse cuatro veces más, y su cobertura saldría cuatro veces más baja.

Antes de enviarlo, dos preguntas a Marcos que cambian la lectura. Una: ¿cada cuánto reponéis? Si el proveedor sirve cada semana, "menos de una semana" es lo normal en muchos productos y no una alarma. Dos: el ritmo de agosto y septiembre es el de verano, el más alto del año; en octubre se vende menos y la cobertura real será mayor. Tu cifra es correcta, pero conviene que Marcos sepa con qué ritmo está hecha.

MJD-307 · ¿Se cumple el 80/20?

Ordenar a los clientes de más a menos ventas y acumular su porcentaje. El cliente en el que el acumulado cruza el 80 % responde a la pregunta:

SQL · EN DUCKDB

```
WITH ventas_cliente AS (
    SELECT p.cliente_maestro_id, sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha > DATE '2026-09-30' - INTERVAL 1 YEAR
      AND p.cliente_maestro_id IS NOT NULL
    GROUP BY p.cliente_maestro_id
),
acumulado AS (
    SELECT cliente_maestro_id, ventas,
           row_number() OVER (ORDER BY ventas DESC, cliente_maestro_id) AS puesto,
           count(*) OVER () AS total_clientes,
           sum(ventas) OVER (ORDER BY ventas DESC, cliente_maestro_id
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
             / sum(ventas) OVER () AS pct_acumulado
    FROM ventas_cliente
)
SELECT puesto AS clientes_necesarios,
       total_clientes,
       round(100.0 * puesto / total_clientes, 1) AS pct_clientes,
       round(100 * pct_acumulado, 1) AS pct_ventas_acumulado
FROM acumulado
WHERE pct_acumulado >= 0.80
ORDER BY puesto
LIMIT 1;
```

RESULTADO

clientes_necesarios	total_clientes	pct_clientes	pct_ventas_acumulado
int64	int64	double	double
487	1092	44.6	80.0

Hacen falta 487 de los 1.092 clientes, el 44,6 %, para llegar al 80 % de las ventas. No es un 80/20, sino más bien un 80/45: las ventas de Marejada están concentradas, pero bastante menos de lo que dice el tópico. El primer quintil del 5.4 lo confirma desde el otro lado: el 20 % de los clientes hace el 59,6 %, no el 80 %.

REVISIÓN DE ELENA

Bien, y fíjate en por qué has hecho bien en usar `ROWS` y un desempate en el acumulado: sin ellos, dos clientes con exactamente las mismas ventas compartirían acumulado (el *peer* de `RANGE`, 6.3) y el "clientes necesarios" podría saltar uno.

Para Íñigo: la cifra desmonta un tópico, y eso siempre interesa en un comité. Pero la lectura útil es otra: la cartera no depende de un puñado de grandes clientes, y eso es una buena noticia (el riesgo está repartido) y un reto (no basta con cuidar a los veinte mayores). Una cartera de clientes clave útil aquí tendría un par de cientos de clientes, no veinte. Esa es la conversación que querrá tener.

MJD-308 · La ficha de cliente para el CRM

Es el ticket que junta casi todo el volumen. Cada dato sale de un sitio distinto, así que una CTE por concepto y una consulta final que los une:

SQL · EN DUCKDB

```
WITH validos AS (
    SELECT * FROM silver.pedidos
    WHERE estado <> 'Cancelado' AND cliente_maestro_id IS NOT NULL
),
historia AS (
    SELECT cliente_maestro_id,
           min(fecha)                                    AS primer_pedido,
           max(fecha)                                    AS ultimo_pedido,
           arg_max(canal, fecha_pedido)                  AS canal_ultimo,
           arg_max(comercial_id, fecha_pedido)           AS comercial_ultimo,
           count(*) FILTER (WHERE fecha > DATE '2026-09-30' - INTERVAL 1 YEAR) AS pedidos_12m
    FROM validos
    GROUP BY cliente_maestro_id
),
ventas_12m AS (
    SELECT v.cliente_maestro_id, sum(l.importe_neto) AS ventas_12m
    FROM validos AS v
    JOIN silver.lineas_pedido AS l ON l.pedido_id = v.pedido_id
    WHERE v.fecha > DATE '2026-09-30' - INTERVAL 1 YEAR
    GROUP BY v.cliente_maestro_id
)
SELECT c.cliente_id, c.nombre_comercial, c.provincia,
       h.primer_pedido, h.ultimo_pedido, h.canal_ultimo,
       co.nombre_completo AS comercial_ultimo,
       h.pedidos_12m,
       round(coalesce(v.ventas_12m, 0), 2) AS ventas_12m,
       rank() OVER (PARTITION BY c.provincia ORDER BY coalesce(v.ventas_12m, 0) DESC) AS
puesto_provincia
FROM silver.clientes AS c
JOIN historia AS h ON h.cliente_maestro_id = c.cliente_id
LEFT JOIN ventas_12m AS v ON v.cliente_maestro_id = c.cliente_id
LEFT JOIN silver.comerciales AS co ON co.comercial_id = h.comercial_ultimo
WHERE NOT c.es_ficha_duplicada
ORDER BY c.provincia, puesto_provincia;
```

RESULTADO

cliente_id int64	nombre_comercial varchar	provincia varchar	…	pedidos_12m int64	ventas_12m double	puesto_provincia int64
11108	Supermercado La Sidra	Asturias	…	138	388547.59	1
10578	Hotel La Judería	Asturias	…	91	195749.96	2
11057	Supermercado La Rula	Asturias	…	79	157043.16	3
11144	Supermercado El Merc…	Asturias	…	75	138296.15	4
·	·	·	…	·	·	·
·	·	·	…	·	·	·
·	·	·	…	·	·	·
10387	Cafetería La Pradera	Palencia	…	1	139.2	56
10179	Hotel El Cuco	Palencia	…	0	0.0	57
10299	Hotel La Estación	Palencia	…	0	0.0	57
11016	Cafetería Albatros	Palencia	…	0	0.0	57

1175 rows (8 shown) 10 columns (6 shown)

1.175 fichas, una por cliente real con pedidos. Son diez columnas; en el libro solo caben seis, en tu terminal ancha las verás todas. Y para el Excel, el `COPY` del Volumen 02 con la consulta dentro (`INSTALL excel; LOAD excel;` si no la tienes cargada en la sesión):

```sql
COPY (
    -- la consulta completa de arriba, sin el punto y coma final
) TO 'entregables/mjd-308_ficha_clientes.xlsx' WITH (FORMAT xlsx, HEADER true);
```

REVISIÓN DE ELENA

Esto es un entregable que se puede poner en producción. Repaso las decisiones que has tomado bien, porque son las que te preguntarán:

- **Una fila por cliente real**: `WHERE NOT c.es_ficha_duplicada` y todos los cálculos por `cliente_maestro_id`. Un CRM con el mismo bar dos veces es lo primero que los comerciales odian.
- **`arg_max` para el canal y el comercial del último pedido**: la forma 3 del 7.4. En el Warehouse de Fabric no existe, y tendrías que usar la forma 2.
- **`LEFT JOIN` con las ventas de 12 meses**, y `coalesce(..., 0)`: un cliente que no ha comprado en un año sigue en el CRM, con cero.

Y una que le añadiría: la fecha de corte en el nombre del fichero o en una columna. Dentro de un mes, alguien abrirá este Excel y no sabrá de cuándo son los "últimos 12 meses".

MJD-309 · Una consulta lenta

Tiene dos problemas, uno de cada tipo del capítulo 9:

1. `year(p.fecha) >= 2026` envuelve la columna en una función: el motor tiene que calcular el año de cada pedido antes de poder descartarlo (9.6). Se cambia por `p.fecha >= '2026-01-01'`.
2. **El puesto con una subconsulta correlacionada**: para cada cliente, cuenta cuántos clientes de su provincia venden más. Son comparaciones de cada cliente con todos los de su provincia. Es exactamente lo que hace `rank()`, con una sola pasada ordenada.

La versión arreglada, con una comprobación de que da lo mismo: se calculan los dos puestos lado a lado y se cuentan las filas en las que no coinciden.

SQL · EN DUCKDB

```
WITH ventas AS (
    SELECT p.cliente_maestro_id, c.provincia, sum(l.importe_neto) AS ventas
    FROM silver.pedidos AS p
    JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
    JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
    WHERE p.estado <> 'Cancelado' AND p.fecha >= '2026-01-01'
    GROUP BY ALL
),
comparacion AS (
    SELECT v.*,
           (SELECT count(*) FROM ventas AS w
            WHERE w.provincia = v.provincia AND w.ventas > v.ventas) + 1 AS puesto_antiguo,
           rank() OVER (PARTITION BY provincia ORDER BY ventas DESC) AS puesto_nuevo
    FROM ventas AS v
)
SELECT count(*) AS clientes,
       count(*) FILTER (WHERE puesto_antiguo <> puesto_nuevo) AS no_coinciden
FROM comparacion;
```

RESULTADO

```
┌──────────┬──────────────┐
│ clientes │ no_coinciden │
│  int64   │    int64     │
├──────────┼──────────────┤
│     1074 │            0 │
└──────────┴──────────────┘
```

Cero diferencias: el `count(*) + 1` de los que venden más **es** la definición de `rank()`, empates incluidos. Ese tipo de comprobación (calcular las dos versiones y contar las diferencias) es la forma de demostrar que una optimización no ha cambiado el resultado.

Y el tiempo, en un MacBook Air M2, ejecutando cada versión tres veces y quedándose con la mejor:

VERSIÓN	MEJOR DE TRES EJECUCIONES (`REAL`)	RESULTADO
Antigua (`year()` y subconsulta correlacionada)	0,009 s	1.074 clientes, suma de control 226.819
Nueva (`fecha >=` y `rank()`)	0,005 s	1.074 clientes, suma de control 226.819

Medidas con la consulta envuelta en un `SELECT count(*), sum(puesto)` para no imprimir 1.074 filas. Casi el doble de rápida.

REVISIÓN DE ELENA

Bien: dos causas identificadas, la demostración de que el resultado no cambia y el tiempo medido, que es lo que te pedí. En DuckDB la ganancia es pequeña en valor absoluto porque los datos son pocos y el motor es listo. Lo importante es el porqué: la versión antigua compara cada cliente con todos los de su provincia, así que su coste crece con el **cuadrado** del número de clientes. Con diez veces más clientes, tardaría unas cien veces más; la de `rank()`, unas diez. En una entrevista, esa frase vale más que el tiempo exacto.

APÉNDICES

Para tener a mano

La chuleta de este volumen y lo que viene en el siguiente.

A. Chuleta de SQL II

B. Qué viene en el Volumen 04

A. Chuleta de SQL II

Subconsultas y CTE

NECESITAS	ESCRIBE
Un valor calculado en un filtro	`WHERE x > (SELECT avg(x) FROM t)`
Lo que no está en otra tabla	`WHERE NOT EXISTS (SELECT 1 FROM t2 WHERE t2.id = t1.id)`. Nunca `NOT IN` si puede haber nulos
Pasos con nombre	`WITH a AS (...), b AS (SELECT ... FROM a) SELECT ... FROM b`
Recorrer una jerarquía	`WITH RECURSIVE j AS (inicio UNION ALL paso que usa j) SELECT ...`
Vista que no se guarda	`CREATE OR REPLACE TEMP VIEW v AS SELECT ...`

Funciones de ventana

NECESITAS	ESCRIBE
Total del grupo en cada fila	`sum(x) OVER (PARTITION BY g)`
Porcentaje del total	`100 * x / sum(x) OVER ()`
Porcentaje con GROUP BY	`100 * sum(x) / sum(sum(x)) OVER ()`
Numerar / ranking	`row_number()`, `rank()`, `dense_rank()` `OVER (PARTITION BY g ORDER BY x DESC)`
Top N por grupo	`QUALIFY rank() OVER (...) <= N` (o CTE + `WHERE`)
Deduplicar	`QUALIFY row_number() OVER (PARTITION BY clave ORDER BY criterio) = 1`
Grupos iguales	`ntile(5) OVER (ORDER BY x DESC)`
Acumulado	`sum(x) OVER (PARTITION BY año ORDER BY mes ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
Media móvil	`avg(x) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Fila anterior / siguiente	`lag(x)`, `lead(x)`, `lag(x, 12)` `OVER (ORDER BY ...)`
Primera / última de la partición	`first_value(x)`, `last_value(x)` con marco `UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`
Valor en el máximo	`arg_max(x, f)` (DuckDB) · `max_by(x, f)` (Databricks)
Ventana reutilizable	`... OVER w ... WINDOW w AS (PARTITION BY ... ORDER BY ...)`

Fechas

NECESITAS	ESCRIBE
Todos los días o meses	`generate_series(DATE 'a', DATE 'b', INTERVAL 1 MONTH) AS t(d)` y `d::DATE`
Combinaciones con huecos	`meses CROSS JOIN grupos LEFT JOIN datos ON mes Y grupo`, y `coalesce(x, 0)`

NECESITAS	ESCRIBE
Interanual sin depender del orden	`LEFT JOIN t AS b ON b.mes = a.mes - INTERVAL 1 YEAR`
Lunes de la semana	`date_trunc('week', f)`
Día de la semana (1 = lunes)	`isodow(f)`
Mediana	`median(x)`

Rendimiento

NECESITAS	ESCRIBE
Medir	`.timer on`
Ver el plan	`EXPLAIN consulta;`
Ver lo que ha pasado	`EXPLAIN ANALYZE consulta;`
Filtrar fechas sin frenar al motor	`f >= '2026-01-01'` en lugar de `year(f) = 2026`
Demostrar que una optimización no cambia nada	Las dos versiones lado a lado y `count(*) FILTER (WHERE a <> b)`

Las trampas de este volumen

TRAMPA	CÓMO SE EVITA
`NOT IN` con un `NULL` en la lista	`NOT EXISTS` o anti-join
Una ventana en el `WHERE`	`QUALIFY` o CTE con la ventana y `WHERE` fuera
Un agregado en la misma consulta que el `QUALIFY`	Filtrar en una subconsulta y contar fuera
`row_number()` con empates	Desempate que no se repita: `ORDER BY x DESC, id`
Acumulado con empates en el orden	`ROWS` y desempate, no el `RANGE` por defecto
`last_value` que devuelve la propia fila	Marco hasta `UNBOUNDED FOLLOWING`, o `max() OVER (PARTITION BY ...)`
`lag()` sobre una serie con huecos	Calendario completo antes de la ventana
Condición del `LEFT JOIN` en el `WHERE`	En el `ON`, si debe conservar las filas sin pareja

B. Qué viene en el Volumen 04

El Volumen 04 deja de responder preguntas sueltas y construye la capa que las responderá todas: **gold**, el modelo en estrella que leerán Power BI en el Bloque 1 y Fabric en el Bloque 2.

- **Hechos y dimensiones**: qué es una tabla de hechos, qué es una dimensión y por qué separarlas.
- **El grano**: la pregunta del Volumen 02 (¿qué es una fila?) convertida en la primera decisión de diseño.
- **La dimensión calendario**, que ya has construido aquí, convertida en tabla permanente.
- **Claves subrogadas** y por qué los modelos no usan los identificadores del ERP.
- **Dimensiones que cambian con el tiempo (SCD)**: qué pasa con las ventas de un cliente cuando cambia de comercial, y cómo guardarlo bien.
- **El script** `sql/03_construir_gold.sql`, reproducible como los de bronze y silver, con sus comprobaciones.

> **ANTES DE PASAR AL VOLUMEN 04**
>
> Tienes que poder hacer esto sin mirar: explicar por qué `NOT IN` falla con nulos, escribir el último pedido de cada cliente con `row_number()` y CTE (sin `QUALIFY`), calcular un porcentaje del total con una ventana, un acumulado que se reinicie cada año y una variación interanual que no dependa de que estén todos los meses. Y tener los nueve tickets en `sql/03-sql-ii/` y en GitHub.